

Contents

E1 Deep-Dive: Tài liệu chuẩn bị phản biện	2
0. E1 là gì, và vì sao thiết kế như vậy	2
Sơ đồ tổng quát toàn bộ E1 (nhìn 1 phút nắm cả hành trình)	3
1. Bản đồ những gì tôi đã làm (theo thời gian)	4
Bản đồ mã nguồn: mỗi đoạn code trong tài liệu này nằm ở file nào, block nào	4
2. M1: Tokenizer BPE 12.000 tự huấn luyện	6
Tôi làm gì	6
Quyết định và lý do	6
Code hoạt động thế nào	6
Phản biện dự kiến	7
3. M2: Pipeline dữ liệu điều kiện 5-slot	7
Tôi làm gì	7
Quyết định và lý do	7
Code hoạt động thế nào	7
Phản biện dự kiến	8
4. M3: Kiến trúc Llama 30M, giải phẫu từng lựa chọn	8
Tôi làm gì	8
Quyết định và lý do, từng thành phần	8
Code hoạt động thế nào	8
Phản biện dự kiến	9
5. M4: Vòng lặp huấn luyện, từng dòng quan trọng	9
Tôi làm gì	9
Quyết định và lý do	9
Code hoạt động thế nào	10
Phản biện dự kiến	10
6. M5: v1 cố ý under-train và Phase 1, lập luận scaling law	10
Hiểu M5 bằng một ẩn dụ (đọc trước khi vào chi tiết)	10
Tôi làm gì	11
Quyết định và lý do	11
Kết quả đọc thế nào	11
Code hoạt động thế nào	11
Phản biện dự kiến	12
7. M6: Phase 2, can thiệp phân bố dữ liệu	12
Tôi làm gì	12
Quyết định và lý do	12
Code hoạt động thế nào (quota cap khi stream)	12
Phản biện dự kiến	12
8. M7: Hệ đo lường, và phát hiện quan trọng nhất về phương pháp	13
Tôi làm gì	13
Từng thước đo nghĩa là gì và vì sao chọn	13
Phát hiện phương pháp luận: nhiều judge	13
Phản biện dự kiến	13
9. M8: Chiến dịch hậu huấn luyện 5 phương pháp (phần dễ bị hỏi xoáy nhất)	13
9.1 DPO (Direct Preference Optimization): null 7.88 vs 8.02	14
9.2 Best-of-N: +0.8, phương pháp DUY NHẤT ăn tiền, đã ship vào app	14
9.3 RAFT (reward-ranked fine-tuning): null 7.60 vs 7.78	14
9.4 Reward model 30M: rút công kiểm định (thí nghiệm phụ trợ)	14
9.5 GRPO-lite (REINFORCE + baseline theo nhóm): null ở ngân sách này (+0.09, n=45)	15
9.6 Distillation từ teacher: Âm -0.37, và vì sao kết quả âm này quý	15
Bảng nhớ nhanh cho phản biện	15
10. M9: 60M, kiểm chứng thuận chiều kết luận	16
Tôi làm gì	16
Quyết định và lý do	16

Kết quả (thuộc số)	16
11. M10: Tích hợp ứng dụng	17
12. E1 dưới lăng kính BÁO CÁO NHÓM: benchmark chung, các phép bóc tách, và cách trả lời về điểm 3.30	17
12.1 Hai tầng đánh giá của nhóm (phải phân biệt được ngay)	17
12.2 Vì sao 3.30 ở vòng chung trong khi 8.96 nội bộ: bốn nguyên nhân, xếp theo trọng số	17
12.3 Ba mức tuân thủ điều kiện: khung khái niệm trung tâm của báo cáo nhóm	18
12.4 E1 trả lời câu hỏi nghiên cứu nào của nhóm	18
12.5 Các chi tiết kỹ thuật nhỏ trong báo cáo nhóm cần biết khi bị soi	18
13. Từ điển thuật ngữ cho người bắt đầu từ số 0	19
A. GenAI trong 5 phút: vòng lặp cốt lõi	19
B. Chữ và số: cách máy “đọc” văn bản	19
C. Bên trong cỗ máy: kiến trúc transformer	19
D. Huấn luyện: các khái niệm vận hành	20
E. Dữ liệu có điều kiện: các khái niệm riêng của E1	21
F. Đánh giá: làm sao biết truyện “tốt”	21
G. Hậu huấn luyện: các cách “dạy thêm” và vì sao khó	21
H. Triển khai: từ trọng số đến ứng dụng	22
14. Ngân hàng câu hỏi phản biện tổng hợp (trả lời trong 2-4 câu)	22
15. Ba mạch kể để mở đầu câu trả lời bất kỳ	23
16. Tóm tắt nhanh M1 -> M9 (kèm định nghĩa term, để ôn cấp tốc)	23
M1 - Tokenizer BPE 12k	23
M2 - Pipeline dữ liệu điều kiện 5-slot	23
M3 - Kiến trúc Llama 30M	23
M4 - Vòng lặp huấn luyện	24
M5 - Chẩn đoán under-training + Phase 1	24
M6 - Phase 2: can thiệp phân bố dữ liệu	24
M7 - Hệ đo lường + phát hiện nhiều judge	24
M8 - Chiến dịch hậu huấn luyện (5 phương pháp)	24
M9 - Scale up 60M (kiểm chứng)	25
Một câu kết cho toàn bộ	25
17. Hai cơ chế judge: judge nội bộ vs judge tổng (vòng chung)	25
Bảng đối chiếu	25
Mỗi judge trả lời câu hỏi gì	25
Vì sao KHÔNG so hai điểm với nhau được	26
Vì sao slm-60m tụt mạnh ở vòng chung (3.30)	26

E1 Deep-Dive: Tài liệu chuẩn bị phản biện

Người thực hiện: Lê Hải Triều (20252611M). Tài liệu nội bộ, dùng để nắm bản chất toàn bộ E1 trước buổi bảo vệ. Đối chiếu với báo cáo nhóm (output/pdf/tinystory-vn-team-report.pdf, mục 2.2) và báo cáo cá nhân (trieulh/report/report.pdf).

0. E1 là gì, và vì sao thiết kế như vậy

Một câu tóm tắt E1: huấn luyện TỪ ĐẦU (khởi tạo ngẫu nhiên, không dùng trọng số có sẵn) một mô hình ngôn ngữ kiểu Llama cỡ 30M rồi 60M tham số trên kho truyện ngụ ngôn TF1-EN-3M, và khảo sát có hệ thống **bốn nhóm can thiệp**: (1) ngân sách token tiền huấn luyện, (2) phân bố dữ liệu, (3) các phương pháp hậu huấn luyện, (4) quy mô tham số.

Vị trí trong nhóm: E1 là hướng duy nhất kiểm soát được TOÀN BỘ pipeline từ tokenizer đến trọng số cuối (E2 cũng from-scratch nhưng họ GPT-2 và tập trung tokenizer/điều kiện hóa; E3-E5 kế thừa mô hình pretrain

sản). Nhờ kiểm soát toàn bộ, E1 trả lời được câu hỏi mà các hướng khác không hỏi được: **cái gì quyết định "sản" chất lượng của một mô hình nhỏ, và sản đó dịch chuyển được bằng gì?**

Câu trả lời sau toàn bộ thí nghiệm (thuộc lòng câu này): > Sản chất lượng do PRETRAINING quyết định (dữ liệu + token + capacity). Năm phương pháp > hậu huấn luyện chi phí thấp không nâng được sản (bốn null, một âm); tìm kiếm lúc suy > luận (best-of-N) khai thác được phần đuôi tốt của phân bố (+0.8 điểm); và khi đầu tư > đúng chỗ (60M, full data, context 1024) thì sản tăng +1.0 điểm đúng như chẩn đoán.

Sơ đồ tổng quát toàn bộ E1 (nhìn 1 phút nắm cả hành trình)

```
TF1-EN-3M (3 triệu truyện ngụ ngôn tiếng Anh, mỗi truyện kèm 5 slot đề bài)
|
v
[M2] LÂM DỮ LIỆU                                [M1] TOKENIZER BPE 12k
lọc 60-320 từ, khử trùng lặp                        học "bảng chữ cái" riêng
ghép: <5 slot> <|story|> truyện <|end|>           từ chính kho truyện
che slot ngẫu nhiên (slot dropout)                  |
|-----|
|
v
[M3] KIẾN TRÚC LLAMA 30M (36.6M tham số, khởi tạo NGẪU NHIÊN)
8 khối transformer, hidden 512, GQA, RoPE, SwiGLU, context 512
|
v
[M4-M5] TIỀN HUẤN LUYỆN (trên Colab T4, checkpoint mỗi 500 bước)
|
|-- v1: cố ý cho ÍT dữ liệu (150k) -----> judge 2.5/10  "bệnh gì?"
|      chẩn đoán: under-training (thiếu token, không thiếu não)
|
|-- Phase 1: THÊM TOKEN (400k x4 epoch) --> judge 6.0    "đúng bệnh!"
|
|-- [M6] Phase 2: SỬA PHÂN BỐ DỮ LIỆU ----> judge 7.0, loss 1.278
|      (chặn khuôn "wise old owl" 28%->10%, hạ dropout moral)
|
v
[M7] HỆ ĐO LƯỜNG
perplexity vs sản | Distinct/Self-BLEU/Flesch/Zipf | LLM-judge 4 trực
+ TỰ ĐO NHIỀU JUDGE (+/-0.4) -> quy tắc: kết luận phải n=45, seed bất cặp
|
v
[M8] CHIẾN DỊCH HẬU HUẤN LUYỆN: "có nâng được điểm mà KHÔNG train lại từ đầu?"
|
|-- DPO 194 cặp ----- NULL (7.88 vs 8.02)
|-- SFT-on-best 42 truyện ----- NULL (7.98 vs 8.02)
|-- RAFT 200 truyện >=9.0 ----- NULL (7.60 vs 7.78)
|-- Reward model 30M ----- RÚT CỔNG (đoán ngang tung đồng)
|-- GRPO-lite 60 bước RL ----- NULL ở budget này (+0.09, n=45)
|-- Distill từ Qwen-4B 600 bài - ÂM (-0.37: bắt chước hỏng giọng)
|-- Best-of-N (sinh 3 chọn 1) -- +0.8 DUY NHẤT ẮN TIỀN -> ship vào app
|
v
KẾT LUẬN CỨ CHẾ: "sản" chất lượng do PRETRAINING quyết định,
không có đường tắt hậu huấn luyện giá rẻ
|
v
```

[M9] KIỂM CHỨNG: SCALE 60M (hidden 768, context 1024, FULL 2.34M truyện)
 judge 7.94 -> 8.96 (+1.0, t=6.53) : ĐÚNG NHƯ CHẨN ĐOÁN

|
v

[M10] ỨNG DỤNG: slm-60m + best-of-N trong web app (~900 token/giây)

Cách đọc sơ đồ khi bị hỏi “em đã làm gì”: đi từ trên xuống, mỗi mũi tên là một quyết định có lý do, mỗi ô kết quả có con số kiểm chứng được. Nhánh M8 toàn null KHÔNG phải thất bại: nó là bằng chứng loại trừ dẫn tới kết luận cơ chế, và M9 xác nhận kết luận đó.

1. Bản đồ những gì tôi đã làm (theo thời gian)

#	Milestone	Kết quả then chốt
M1	Tokenizer BPE 12k tự huấn luyện	vocab nhỏ giữ embedding ~12% tham số
M2	Pipeline dữ liệu điều kiện 5-slot	format train = format inference, loss-mask
M3	Kiến trúc Llama 30M (36.6M)	8 khối, hidden 512, GQA 8/2, seq 512
M4	Vòng lặp huấn luyện + WSD + resume	fp16 T4, checkpoint 500 bước, sống qua disconnect
M5	v1 cố ý under-train, chẩn đoán, Phase 1	judge 2.5 lên 6.0 chỉ bằng THÊM TOKEN
M6	Phase 2: can thiệp phân bố dữ liệu	owl 90% xuống 23%, judge 7.0, loss 1.278
M7	Hệ đo lường + tự đo nhiều judge	phát hiện nhiễu +-0.4, protocol n=45 seed bắt cặp
M8	Chiến dịch hậu huấn luyện 5 phương pháp	DPO/SFT-best/RAFT/GRPO null, distill âm, best-of-N +0.8
M9	Scale-up 60M, full TF1, seq 1024	judge 8.96 (n=45, t=6.53), sản tăng +1.0
M10	Tích hợp app	best-of-N endpoint, registry, quick evaluation

Mỗi milestone dưới đây trình bày theo khung: **Tôi làm gì / Quyết định và lý do / Code hoạt động thế nào / Câu hỏi phản biện dự kiến.**

Bản đồ mã nguồn: mỗi đoạn code trong tài liệu này nằm ở file nào, block nào

Hai notebook Colab đã được kéo từ Drive về repo (kèm nguyên output cell làm bằng chứng chạy): `trieulh/notebooks/pretrain_slm_tf1_clean.ipynb` (30M, 20 cell) và `trieulh/notebooks/pretrain_slm_60m.ipynb` (60M, 4 cell). Lưu ý kiến trúc: notebook 60M chỉ là LAUNCHER (mount Drive, clone repo, gọi script); toàn bộ logic 60M nằm trong `trieulh/scripts/colab_pretrain_60m.py`. Notebook 30M thì chứa logic train trực tiếp trong cell.

Đoạn code trong deep-dive	File	Vị trí cụ thể
M1: train BPE 12k	<code>trieulh/scripts/train_tokenizer.py</code>	hàm <code>train_bpe()</code> ; được gọi trong bước build corpus (notebook 30M cell 3, Step 1)
M2: parse 5 slot từ prompt TF1	<code>trieulh/scripts/tf1_pretrain/parse.py</code>	hàm <code>parse_slots()</code>

Đoạn code trong deep-dive	File	Vị trí cụ thể
M2: slot dropout + ghép chuỗi train	trieulh/scripts/tf1_pretrain/for	<code>apply_dropout(), build_training_text(), length_bucket()</code>
M2: lọc 60-320 từ, dedupe, stream corpus	trieulh/scripts/prepare_tf1_pretrain	<code>build_record(), _write_split()</code>
M2: encode + loss-mask -100	notebook 30M cell 7 (Step 3)	hàm <code>encode()</code> ; bản 60M: <code>build_or_restore_corpus()</code> trong <code>colab_pretrain_60m.py</code>
M3: dựng LlamaConfig 30M từ random init	notebook 30M cell 9 (Step 4)	trong hàm <code>train(size);</code> hyperparam khai báo ở cell 5 (Step 2)
M4: hàm <code>wsd()</code> + LambdaLR	notebook 30M cell 9 ; bản CLI: <code>trieulh/scripts/colab_phase2.py</code> ; bản 60M: <code>colab_pretrain_60m.py</code>	cùng công thức <code>warmup/stable/decay</code> ở cả ba nơi
M4: collator pad động	notebook 30M cell 7 ; <code>colab_phase2.py</code> ; <code>colab_pretrain_60m.py</code>	hàm <code>collator(features)</code>
M4: checkpoint 500 bước + auto-resume	notebook 30M cell 9 (<code>TrainingArguments + resume_from_checkpoint</code>); <code>colab_pretrain_60m.py</code>	<code>save_steps=500,</code> <code>save_total_limit=2</code>
M5-M6: chạy Phase 2 resume	notebook 30M cell 11 (<code>trainer30 = train("30M")</code>); bản <code>headless</code> : <code>colab_phase2.py main()</code>	
M6: quota cap “wise old owl” 10%	trieulh/scripts/prepare_tf1_pretrain	<code>vang_diem</code> phrase_written trong <code>_write_split()</code> ; tham số <code>--cap-phrase/--cap-frac</code> truyền từ notebook cell 3 / <code>colab_phase2.py</code>
M7: harness gom số liệu + 3 figure + verdict	notebook 30M cell 13-17 (Step 5b)	<code>collect_analysis()</code> + các cell figure; metric nền: <code>app/metrics.py</code> , <code>app/perplexity.py</code>
M7: LLM-judge 4 trục	<code>app/judge.py</code>	<code>evaluate()</code> + <code>AXIS_RUBRIC</code>
M7: protocol n=45 seed bắt cặp, resume-safe	trieulh/scripts/big_judge_eval.py (60M: <code>sixty_judge_eval.py</code> ; RAFT: <code>raft_judge_eval.py</code>)	progress file jsonl chấm đến đâu lưu đến đó
M8 DPO: cấu hình + ref model + ppl guard	trieulh/scripts/dpo_train_local.py	<code>DPOConfig(beta=0.1, lr=5e-6), DPOTrainer(model, ref_model=...)</code>
M8: sinh cặp preference	trieulh/scripts/gen_preference_pairs.py	<code>make_pair()</code> , lọc <code>margin >= 1.0</code>
M8: thăm dò headroom best-of-N	trieulh/scripts/headroom_probe.py	<code>K=3, temps [0.5, 0.8, 1.1]</code>
M8: best-of-N trong app	<code>app/main.py</code>	nhánh <code>if req.best_of_n > 1</code> trong <code>generate_stream</code> + hàm <code>pick_best_index()</code>
M8 RAFT: corpus ngưỡng 9.0 + SFT	trieulh/scripts/raft_harvest.py, <code>raft_gen_corpus.py</code> , <code>sft_best_local.py</code>	SFT dùng chung trainer với <code>loss-mask</code>
M8 RM: 2 biến thể + cổng kiểm định	trieulh/scripts/rm_train.py (MSE), <code>rm_train_pairwise.py</code> (Bradley-Terry)	class <code>RewardModel</code> , hàm <code>spearman()/validate()</code>

Đoạn code trong deep-dive	File	Vị trí cụ thể
M8 GRPO: 6 bước một vòng	trieulh/scripts/grpo_train.py	rollout(), reward_fn(), chuẩn hóa advantage, logprobs_mean(), phạt KL, checkpoint+state.json
M8 Distill: teacher sinh corpus	trieulh/scripts/distill_gen_corpus.py	system prompt ràng buộc văn phong, lọc <= 400 token
M9 60M: toàn bộ pipeline	trieulh/scripts/colab_pretrain_60m.py	os.path.join(drive_root(), build_or_restore_corpus(pack int16), class PackedDS, wsd(), callback DriveLog; launcher: notebook 60M cell 2-4
M10: hoàn thiện truyện (trim, done_reason)	app/textproc.py (trim_to_last_sentence), app/prompt_en.py (LENGTH_NUM_PREDICT)	

Khi phản biện yêu cầu “mở code ra xem”: mở đúng file/cell theo bảng này; với 30M thì pretrain_slm_tf1_clean.ipynb còn giữ nguyên OUTPUT của lần chạy thật (bảng loss, 3 figure inline) nên vừa là code vừa là bằng chứng.

2. M1: Tokenizer BPE 12.000 tự huấn luyện

Tôi làm gì

Huấn luyện một tokenizer BPE byte-level riêng trên chính kho truyện, vocab 12.000, thêm 3 token đặc biệt <|story|>, <|end|>, <|pad|> (trieulh/scripts/train_tokenizer.py).

Quyết định và lý do

- **Vì sao KHÔNG dùng tokenizer có sẵn (GPT-2 50k, Llama 32k)?** Bảng embedding chiếm vocab x hidden tham số. Với hidden 512: vocab 50k tốn 25.6M tham số CHỈ cho embedding, gần bằng cả phần thân 30M. Vocab 12k chỉ tốn 6.1M (~17%), phần còn lại dồn cho các khối transformer thực sự “suy nghĩ”. Ở mô hình lớn tỉ lệ này không đáng kể, ở 30M nó quyết định.
- **Vì sao 12.000 mà không nhỏ hơn?** Miền hẹp (truyện thiếu nhi, từ vựng đơn giản) nên 12k đã phủ tốt: trung bình ~1.3-1.5 token/từ, câu không bị băm vụn. Nhỏ hơn nữa (4k) thì chuỗi dài ra, tốn context, và attention phải học ghép vắn thay vì ghép ý.
- **Vì sao byte-level BPE?** Không bao giờ gặp <unk> với input lạ (mọi byte đều mã hóa được), tương thích hệ sinh thái GPT-2/llama.cpp khi convert GGUF về sau.

Code hoạt động thế nào

```
tok = Tokenizer(models.BPE(unk_token="<|unk|>"))
tok.pre_tokenizer = pre_tokenizers.ByteLevel(add_prefix_space=False)
trainer = trainers.BpeTrainer(vocab_size=12000,
    special_tokens=["<|unk|>", PAD, SEP, END])
tok.train_from_iterator(texts, trainer=trainer)
```

- ByteLevel pre-tokenizer: tách văn bản thành byte trước khi học merge, nên vocab học được là các “mảnh byte” phổ biến (thường trùng từ/tiền tố trong miền truyện).
- BPE (Byte-Pair Encoding) học lặp: đếm cặp ký hiệu kề nhau xuất hiện nhiều nhất, gộp thành ký hiệu mới, lặp cho đến khi đủ 12.000 mục. Kết quả: từ phổ biến (“the”, “fox”, “forest”) thành 1 token, từ hiếm bị tách thành nhiều mảnh.
- Token đặc biệt được khai báo TRƯỚC khi train để không bao giờ bị BPE tách đôi: <|story|> đánh dấu ranh giới điều kiện/truyện, <|end|> để mô hình HỌC CÁCH DỪNG (đây là eos), <|pad|> để đệm batch.

Phản biện dự kiến

- “BPE là gì, khác word-level và char-level chỗ nào?” Trung gian giữa hai cực: char-level chuỗi rất dài, word-level nổ vocab với từ hiếm; BPE học đơn vị “dưới từ” theo tần suất nên cân bằng độ dài chuỗi và độ phủ.
 - “Nếu dùng vocab 50k thì sao?” Tham số embedding phình to chiếm chỗ của thân mô hình; với cùng ngân sách 30M, thân mỏng đi và chất lượng giảm. Đây là trade-off đặc thù mô hình nhỏ.
-

3. M2: Pipeline dữ liệu điều kiện 5-slot

Tôi làm gì

Biến mỗi bản ghi TF1 (prompt 5 slot + truyện) thành chuỗi huấn luyện có điều kiện, kèm lọc chất lượng, dedupe, slot dropout (`trieulh/scripts/prepare_tf1_pretrain.py` + `tf1_pretrain/format.py`, `parse.py`).

Quyết định và lý do

- **Format train TRÙNG format inference.** `build_training_text` gọi đúng hàm `build_fable_prompt` mà app dùng lúc sinh. Lý do: mô hình nhỏ rất nhạy với distribution shift; nếu train một kiểu prompt, serve kiểu khác thì mất điểm oan. Đây là quyết định “một nguồn sự thật cho prompt”.
- **Loss-mask phần điều kiện (nhãn -100).** Mục tiêu tối ưu là $\max \log P(\text{truyện} \mid \text{điều kiện})$, KHÔNG phải học thuộc cách viết prompt. Nếu tính loss cả phần điều kiện, mô hình tốn dung lượng học cấu trúc template thay vì học kể chuyện.
- **Slot dropout (mặc định $p=0.3$, $p_{\text{all}}=0.05$).** Người dùng thật có thể bỏ trống slot bất kỳ. Che ngẫu nhiên từng slot lúc train để mô hình quen mọi tổ hợp; 5% số mẫu che sạch toàn bộ để hỗ trợ chế độ sinh tự do.
- **Lọc 60-320 từ.** Dưới 60 thường là mẫu cụt; trên 320 vượt quá phần context dành cho truyện ở seq 512 (truyện dài sẽ bị cắt giữa chừng lúc train, dạy mô hình “bỏ lửng”).
- **Dedupe theo prompt_hash.** TF1 là dữ liệu tổng hợp, có trùng lặp; trùng lặp làm mô hình học vẹt và làm sai lệch thống kê đánh giá.

Code hoạt động thế nào

```
def build_training_text(slots, fable, length):
    cond = build_fable_prompt(slots..., LENGTH_HINT_EN[length])
    prefix = cond + "\n"
    text = prefix + SEP + fable.strip() + END
    return text, len(prefix)  # cond_len tính bằng KÝ TỰ
```

Chuỗi cuối: <điều kiện>\n<|story|><truyện><|end|>. Hàm trả thêm `cond_len` (vị trí bắt đầu <|story|>) để bước encode biết mask đến đâu:

```
def encode(row):
    ids = tok(row["text"], truncation=True, max_length=512)["input_ids"]
    n_cond = min(len(tok(row["text"][:row["cond_len"]])["input_ids"]), len(ids))
    return {"input_ids": ids, "labels": [-100]*n_cond + ids[n_cond:]}
```

- Encode cả chuỗi một lần, rồi encode RIÊNG phần prefix để đếm nó chiếm bao nhiêu token; bấy nhiêu nhãn đầu gán -100. PyTorch `CrossEntropyLoss(ignore_index=-100)` sẽ bỏ qua các vị trí này khi tính loss và gradient.
- `apply_dropout` che slot: xác suất p_{all} che hết (mẫu free-gen), ngược lại che từng slot độc lập với xác suất p hoặc $p_{\text{overrides}}[\text{slot}]$ (Phase 2 hạ teaching/outcome xuống 0.15, xem M6).
- `length_bucket` gán nhãn short/medium/long theo số từ THẬT của truyện, để gợi ý độ dài trong prompt khớp với đầu ra: mô hình học liên kết “gợi ý ngắn” với truyện ngắn.

Phản biện dự kiến

- “-100 có ý nghĩa gì?” Quy ước ignore_index của PyTorch: vị trí nhãn -100 không đóng góp vào loss, gradient bằng 0 tại đó. Mô hình vẫn ĐỌC (attend) phần điều kiện, chỉ không bị chấm điểm khi dự đoán nó.
- “Không mask thì sao?” Vẫn train được, nhưng một phần đáng kể ngân sách gradient dồn vào việc dự đoán template lặp đi lặp lại, và ppl đo được sẽ trộn lẫn hai thứ không cùng bản chất.
- “Slot dropout khác gì dropout thường?” Dropout thường tắt NEURON ngẫu nhiên (regularize trọng số); slot dropout tắt TRƯỜNG DỮ LIỆU trong prompt (augmentation cấp dữ liệu, mô phỏng phân bố input thật).

4. M3: Kiến trúc Llama 30M, giải phẫu từng lựa chọn

Tôi làm gì

Tự cấu hình decoder-only kiểu Llama bằng LlamaConfig của HF: 8 khối, hidden 512, FFN 2048 (SwiGLU), 8 query head / 2 KV head (GQA), RoPE, RMSNorm, tied embedding, seq 512, vocab 12k. Tổng 36.6M tham số.

Quyết định và lý do, từng thành phần

- **Decoder-only autoregressive:** bài toán là sinh chuỗi; mục tiêu MLE $\log P(x) = \sum_i \log P(x_i | x_{<i})$ (chain rule). Mỗi vị trí chỉ nhìn quá khứ (attention nhân quả có mask tam giác).
- **Vì sao “kiểu Llama” thay vì GPT-2?** Bộ ba RMSNorm + RoPE + SwiGLU là recipe hiện đại đã được kiểm chứng hội tụ ổn hơn ở cùng ngân sách:
 - **RMSNorm** thay LayerNorm: chỉ chuẩn hóa theo norm (không trừ mean, không bias), rẻ hơn và ổn định tương đương. Công thức: $x / \text{rms}(x) * g$ với $\text{rms}(x) = \sqrt{\text{mean}(x^2)}$.
 - **RoPE (Rotary Position Embedding)** thay position embedding học được: mã hóa vị trí bằng phép XOAY vector query/key theo góc tỉ lệ vị trí; tích vô hướng q.k khi đó chỉ phụ thuộc KHOẢNG CÁCH tương đối giữa hai token. Không tốn tham số, ngoại suy độ dài tốt hơn.
 - **SwiGLU FFN:** thay $W2 * \text{gelu}(W1 \cdot x)$ bằng $W2 (\text{swish}(W1 \cdot x) * (W3 \cdot x))$, cơ chế cổng (gating) cho FFN biểu diễn tốt hơn ở cùng cỡ; Llama/PaLM đều dùng.
- **GQA 8 query / 2 KV head:** các query head chia sẻ chung 2 bộ key-value. Giảm tham số attention và thu nhỏ KV-cache lúc suy luận (~4 lần) mà chất lượng gần như MHA đầy đủ. Trade-off hợp lý cho mô hình chạy laptop.
- **Tied embedding (chia sẻ ma trận nhúng vào và chiếu ra):** tiết kiệm nguyên một bảng $12000 \times 512 \sim 6.1\text{M}$ tham số; về ý nghĩa, buộc “không gian đọc” và “không gian viết” của từ trùng nhau, một regularizer tự nhiên với mô hình nhỏ.
- **Vì sao 30M?** Đủ nhỏ để (a) train trọn trên T4 free trong vài giờ, (b) suy luận ~950 token/giây trên laptop, (c) đúng tinh thần câu hỏi nghiên cứu “nhỏ đến mức nào vẫn dùng được”. TinyStories chứng minh dải 10-60M đã kể chuyện mạch lạc được nếu dữ liệu hẹp.
- **Vì sao seq 512?** Truyền mục tiêu 60-320 từ ~ 100-460 token + prompt ~80-110 token: 512 vừa khít, và chi phí attention là $O(n^2)$ nên seq ngắn tiết kiệm đáng kể compute. Đây cũng là “trần kiến trúc” tôi thừa nhận trong báo cáo, và là lý do 60M nâng lên 1024.

Code hoạt động thế nào

Toàn bộ M3 chỉ gói trong 2 lệnh (trieulh/scripts/colab_pretrain_60m.py; notebook 30M cell 9 cùng cấu trúc, số nhỏ hơn):

```
cfg = LlamaConfig(  
    vocab_size=12000,           # số token trong tokenizer BPE (M1)  
    hidden_size=768,           # 512 ở 30M / 768 ở 60M: độ rộng vector chạy trong máy  
    num_hidden_layers=8,       # 8 khối transformer xếp chồng  
    num_attention_heads=12,    # số query head (8 ở 30M)  
    num_key_value_heads=4,     # số KV head (2 ở 30M) -> ít hơn query head = GQA
```



```

intermediate_size=2048,      # kích thước FFN (SwiGLU) bên trong mỗi khối
max_position_embeddings=1024, # context: 512 ở 30M / 1024 ở 60M
tie_word_embeddings=True,    # dùng CHUNG bảng embedding cho chiều đọc vào và chiều ra logit
eos_token_id=<id của |end|>, # token báo "hết truyện" -> model học cách DỪNG
pad_token_id=<id của |pad|>,
)
model = LlamaForCausalLM(cfg) # dựng model theo tờ khai, TRỌNG SỐ NGẪU NHIÊN

```

Hai điểm cần nói được: - LlamaConfig là “tờ khai kiến trúc”: chỉ liệt kê SỐ (mấy khối, rộng bao nhiêu, GQA mấy head). Các thành phần RoPE, RMSNorm, SwiGLU KHÔNG phải khai vì chúng là mặc định của họ Llama trong thư viện transformers. - LlamaForCausalLM(cfg) dựng ra model với **trọng số khởi tạo ngẫu nhiên** - đây chính là nghĩa của “from-scratch”: tại dòng này model chưa biết cả tiếng Anh, mọi năng lực đều học được qua vòng train ở M4.

Phản biện dự kiến

- “Attention làm gì, nói ngắn gọn?” Mỗi token tạo query/key/value; điểm attention $\text{softmax}(qk^T/\sqrt{d})$ quyết định token hiện tại “đọc” các token trước đó với trọng số bao nhiêu, rồi tổng hợp value theo trọng số đó. Mask nhân quả chặn nhìn tương lai.
- “GQA mất gì so với MHA?” Ít góc nhìn key-value độc lập hơn; ở mô hình rất lớn có thể thiệt nhẹ chất lượng, nhưng thực nghiệm (Ainslie 2023) cho thấy gần tương đương, đổi lấy cache nhỏ và suy luận nhanh.
- “36.6M đếm từ đâu?” Embedding $12k \times 512 = 6.1M$ (tied nên tính một lần); mỗi khối: attention ($q, o: 512 \times 512 \times 2$; $k, v: 512 \times 128 \times 2$) $\sim 0.66M$ + SwiGLU (3 ma trận 512×2048) $\sim 3.1M$; 8 khối $\sim 30.5M$; cộng norm $\sim 36.6M$.

5. M4: Vòng lặp huấn luyện, từng dòng quan trọng

Tôi làm gì

Huấn luyện bằng HF Trainer với optimizer/scheduler tự cấu hình, trên Colab T4 fp16, checkpoint mỗi 500 bước lên Google Drive, tự resume (notebook 30M + colab_phase2.py, sau này colab_pretrain_60m.py).

Quyết định và lý do

- **AdamW, betas (0.9, 0.95), weight decay 0.1, grad clip 1.0:** bộ số “chuẩn cộng đồng” cho pretrain LM (Llama/GPT-3 dùng tương tự). beta2=0.95 thay 0.999: trung bình động phương sai gradient “quên nhanh hơn”, phản ứng kịp khi phân bố gradient đổi trong pretrain từ đầu; wd 0.1 regularize; clip 1.0 chặn bước nhảy gradient hiếm gặp làm hỏng trọng số (fp16 càng cần).
- **Peak LR 3e-3, cao bất thường?** Đúng với mô hình NHỎ: LR tối ưu tăng khi mô hình nhỏ đi (muP/kinh nghiệm TinyStories); 30M chịu được 3e-3 với warmup + clip. Bằng chứng thực nghiệm của tôi: grad-norm ổn định 0.1-0.3 suốt run, không spike.
- **Lịch LR Warmup-Stable-Decay (WSD) thay cosine:**
 - warmup 2%: LR tăng dần từ 0, tránh phá vỡ trọng số ngẫu nhiên bằng bước quá lớn khi thống kê Adam chưa ổn định.
 - stable: giữ ĐỈNH phẳng (khác cosine tụt dần liên tục). Lợi ích then chốt: **có thể kéo dài hoặc dừng sớm mà không phải train lại**, chỉ cần “dời điểm decay”. Tôi đã dùng đúng tính chất này hai lần: Phase 2 resume từ bước 1800 chạy tiếp, và 60M rút 15000 xuống 10000 bước giữa chừng theo quyết định thời gian.
 - decay 15-20% cuối: LR hạ về 0, trọng số “kết tinh” về đáy cục bộ, loss rơi thêm một nấc rõ (quan sát được cả ở 30M lẫn 60M: 1.231 xuống 1.058 phần lớn nhờ decay).
- **fp16:** T4 không hỗ trợ bf16; fp16 kèm loss-scaling của HF đủ ổn với clip 1.0.
- **Batch hiệu dụng 128 chuỗi = per-device 32 x grad accum 4 (30M) hoặc 16 x 8 (60M):** VRAM T4 16GB giới hạn batch vật lý; gradient accumulation cộng dồn gradient của nhiều micro-batch trước khi bước optimizer, mô phỏng batch lớn để gradient bớt nhiễu.

- **Checkpoint 500 bước + auto-resume:** hạ tầng free bị thu hồi bất kỳ lúc nào (đã xảy ra nhiều lần thật). save_total_limit=2 tránh đầy Drive. `trainer.train( resume_from_checkpoint=... )` khôi phục CẢ optimizer state (moment Adam), scheduler, bước hiện tại và RNG, nên đường loss nối liền mạch, không phải “train lại từ trọng số”.

Code hoạt động thế nào

```
def wsd(step):
    warm, dec = int(0.02*STEPS), int(0.15*STEPS)
    if step < warm:          return step / warm           # dốc lên
    if step > STEPS - dec:    return (STEPS - step) / dec  # dốc xuống
    return 1.0              # đỉnh phẳng
scheduler = LambdaLR(optimizer, wsd)  # nhân hệ số này với peak LR 3e-3

def collator(features):
    m = max(len(f["input_ids"]) for f in features)  # độ dài lớn nhất batch
    return {
        "input_ids":      pad tới m bằng pad_token,
        "labels":          pad tới m bằng -100,      # phần đệm không tính loss
        "attention_mask": [1]*len + [0]*(m-len),      # attention bỏ qua phần đệm
    }
```

- Collator pad ĐỘNG theo batch (không pad cứng 512) nên tiết kiệm compute với batch toàn truyện ngắn.
- `TrainingArguments(lr_scheduler_type="constant")` + optimizer/scheduler truyền tay: vô hiệu scheduler mặc định của Trainer để WSD tự quản.
- Với 60M, dataset không còn là list Python (2.34M truyện nổ RAM) mà là **mảng numpy int16 đóng gói phẳng** + bảng offset:

```
class PackedDS(torch.utils.data.Dataset):
    def __getitem__(self, i):
        ids = tokens[offsets[i]:offsets[i+1]].astype(np.int64).tolist()
        nc = int(condlens[i])
        return {"input_ids": ids, "labels": [-100]*nc + ids[nc:]}
```

int16 đủ vì vocab 12000 < 32767; 934M token x 2 byte ~ 1.9GB nằm gọn RAM, `mmap_mode="r"` cho phép đọc lười từ đĩa. `condlens` lưu SẴN số token điều kiện của từng mẫu để khỏi tokenize lại prefix mỗi epoch.

Phản biện dự kiến

- “Loss 1.058 nghĩa là gì?” Cross-entropy trung bình mỗi token (đơn vị nat). Trục quan qua perplexity: $e^{1.058} \sim 2.88$, tức trung bình mô hình phân vân giữa ~2.9 lựa chọn token tương đương tại mỗi bước.
- “Vì sao WSD hơn cosine trong bối cảnh của bạn?” Cosine gắn chặt lịch với TỔNG số bước định trước; đổi tổng số bước là phải train lại. WSD tách “học” và “kết tinh”, cho phép resume/kéo dài/dừng sớm, đúng nhu cầu hạ tầng free hay đứt.
- “Gradient accumulation có tương đương batch lớn thật không?” Về gradient trung bình: có (tổng gradient trước khi step). Khác biệt phụ: batch-norm thì không tương đương, nhưng transformer dùng RMSNorm theo chiều feature nên không ảnh hưởng.

6. M5: v1 cố ý under-train và Phase 1, lập luận scaling law

Hiểu M5 bằng một ẩn dụ (đọc trước khi vào chi tiết)

Hình dung dạy một đứa trẻ thông minh (bộ não = 30M tham số, đã có sẵn) viết văn. Có hai cách nó có thể kém: (a) **não đủ nhưng đọc quá ít sách** (under-training), hay (b) **não quá bé** dù đọc bao nhiêu cũng không khá. Hai bệnh này chữa khác nhau hoàn toàn: bệnh (a) cho đọc thêm sách là khỏi; bệnh (b) phải đổi não to hơn (tốn kém).

M5 là bước CHẨN ĐOÁN xem đứa trẻ mắc bệnh nào. Tôi cố ý cho nó đọc RẤT ÍT (v1: 150k truyện) □ viết dở (2.5/10). Rồi CHỈ tăng lượng đọc, giữ nguyên bộ não (Phase 1: 400k truyện đọc 4 lượt) □ viết khá hẳn (6.0/10). Kết luận: đứa trẻ mắc bệnh (a), thiếu sách chứ không thiếu não. Đây là lý do bước nhảy 2.5 □ 6.0 quan trọng: nó chứng minh hướng đầu tư đúng (thêm dữ liệu) trước khi tốn tiền đổi model to.

Vì sao phải cố ý làm dở ở v1? Nếu tôi khởi đầu bằng cấu hình tốt nhất và ra kết quả kém, sẽ không biết lỗi cho não, sách hay cách dạy. Cố ý cô lập MỘT biến (lượng sách) biến việc chẩn đoán thành một thí nghiệm sạch.

Tôi làm gì

Chạy v1 với 150k truyện / 900 bước (~1.7 token/tham số) làm baseline chẩn đoán: judge 2.5/10. Sau đó Phase 1: 400k truyện unique, 1800 bước (~600M token qua 4 epoch): loss 1.447, ppl 4.18, judge 6.0.

Quyết định và lý do

- **Vì sao cố ý train thiếu?** Để tách bạch nguyên nhân. Nếu khởi đầu bằng cấu hình “tốt nhất có thể” và kết quả kém, không biết tại kiến trúc, dữ liệu hay ngân sách. v1 kém đúng như scaling law dự đoán -> giả thuyết “thiếu token, không thiếu capacity” trở thành thử KIỂM CHỨNG ĐƯỢC bằng một can thiệp duy nhất (thêm token, giữ nguyên kiến trúc).
- **Chinchilla (~20 token/tham số):** điểm cân bằng compute-tối-ưu giữa N (tham số) và D (token). 30M cần ~700M token; v1 chỉ cấp 1.7 token/tham số, thiếu hơn 10 lần.
- **Muennighoff (data-constrained):** khi dữ liệu unique có hạn, lặp tới ~4 epoch gần như tương đương dữ liệu mới; quá 4 epoch lợi ích giảm nhanh. Đây là phép tôi dùng để 400k truyện (~150M token unique) “kéo” thành ~600M token huấn luyện hợp lệ.
- **Kaplan power-law làm công cụ CHẨN ĐOÁN:** vẽ loss theo bước trên trục log-log; nếu thẳng ($R^2 \sim 0.96$) nghĩa là run đang ở chế độ lũy thừa, chưa plateau, thêm token còn lãi. Tôi dùng đồ thị này hai lần: quyết định Phase 1 và sau này quyết định scale 60M.

Kết quả đọc thế nào

2.5 lên 6.0 CHỈ bằng thêm token là bước nhảy lớn nhất toàn E1. Bài học phát biểu được: “ở quy mô nhỏ, dữ liệu và ngân sách token quyết định, kiến trúc là thứ yếu”. Một chỉnh nhỏ khác cùng giai đoạn: repeat_penalty 1.3 xuống 1.1 (+0.2 điểm) vì mức phạt lặp cao trừng phạt cả TÊN NHÂN VẬT khiến nhân vật bị đổi giữa truyện (entity drift): minh họa sampling cũng là “tham số chất lượng” nhưng biên độ nhỏ hơn dữ liệu nhiều.

Code hoạt động thế nào

M5 không có code RIÊNG - nó là cách DÙNG cùng một vòng train ở M4 với hai cấu hình dữ liệu khác nhau. “Ngân sách token” chỉ là hai con số trong ô hyperparameter (notebook 30M cell 5):

```
SEQ_LEN      = 512
TRAIN_N      = 400_000      # số truyện dùng cho Phase 1 (v1 chỉ 150k)
STEPS        = 1800         # số bước huấn luyện (v1 chỉ 900)
BATCH_SIZE, GRAD_ACCUM = 32, 4      # batch hiệu dụng = 128 chuỗi
# -> token đã học ~ STEPS x 128 x (số token/chuỗi) ~ 600M cho Phase 1
```

Đổi từ v1 sang Phase 1 = tăng TRAIN_N (150k -> 400k) và STEPS (900 -> 1800), GIỮ NGUYÊN khối ARCH["30M"] (hidden/layers/heads y hệt). Đó chính là “chỉ thêm token, không đổi kiến trúc”.

Công cụ chẩn đoán scaling law (kiểm tra loss có nằm trên đường power-law không) nằm trong harness dashboard, notebook 30M cell 14:

```
# fit đường thẳng cho log(loss) theo log(step) sau warmup
xs = np.log(steps_after_warmup)
ys = np.log(losses_after_warmup)
slope, intercept = np.polyfit(xs, ys, 1)      # hệ số góc = số mũ power-law
r2 = <độ khớp của đường thẳng>               # R^2 = 0.96 -> khớp tốt, chưa plateau
```

Ý nghĩa: nếu các điểm loss (trên trục log-log) nằm gần một đường thẳng, run đang ở “chế độ lũy thừa” mà scaling law dự đoán - tức thêm token vẫn còn kéo loss xuống. Nếu đường cong đã bệt ngang (plateau) thì thêm token vô ích, phải đổi hướng. Đồ thị này cho $R^2 = 0.96$ và chưa bệt, nên tôi tự tin quyết định Phase 1 (và sau này 60M).

Phản biện dự kiến

- “Scaling law phát biểu gì?” Test loss giảm theo lũy thừa của N, D, C: $L \sim (N_0/N)^a + (D_0/D)^b + L_{\text{vô hạn}}$. Hệ quả thực dụng: biết mình đang bị chặn bởi về nào thì đầu tư đúng về đó.
- “Sao không train luôn Chinchilla đủ 20 tok/param ngay từ đầu?” Hạn mức GPU free theo phiên; và về phương pháp, đi từng nấc cho phép quy mỗi mức tăng chất lượng về đúng một nguyên nhân.

7. M6: Phase 2, can thiệp phân bố dữ liệu

Tôi làm gì

Phát hiện template collapse (“wise old owl” 28% trong dữ liệu nhưng ~90% trong truyện sinh), build corpus v2: giới hạn truyện chứa cụm này còn 10%, hạ slot-dropout của teaching/outcome từ 0.30 xuống 0.15; resume từ checkpoint 1800 train tiếp đến 3600. Kết quả: owl-rate sinh 23%, judge 7.0, loss 1.278, ppl 3.56.

Quyết định và lý do

- **Vì sao mode bị khuếch đại?** Sampling lấy mẫu từ phân bố mô hình; mô hình lại làm TRƠN phân bố dữ liệu về phía mode mạnh nhất (mass của các biến thể hiếm dồn về khuôn phổ biến). 28% trong data thành 90% khi sinh là hiện tượng kinh điển của mô hình nhỏ trên dữ liệu tổng hợp lặp khuôn.
- **Vì sao sửa Ở DỮ LIỆU thay vì phạt lúc sampling?** Đã thử hướng sampling (repeat_penalty) và thấy tác dụng phụ (entity drift). Cắt tại nguồn: cap tỉ lệ mẫu chứa cụm ở 10% bằng quota khi stream corpus, mô hình không còn thấy khuôn đủ dày để nghiện. Kết quả 23% (thấp hơn cả prior 28%) chứng minh cơ chế đúng.
- **Vì sao hạ dropout teaching/outcome?** Chẩn đoán định tính: mô hình hay TỰ BỊA bài học thay vì theo bài học yêu cầu. Nguyên nhân khả dĩ: 30% mẫu train bị che slot teaching nên mô hình học “moral là thứ tự sinh cũng được”. Hạ xuống 0.15: mô hình thấy moral trong điều kiện thường xuyên hơn, học BẮM theo nó.
- **Vì sao resume thay vì train mới?** Tiết kiệm nửa ngân sách và giữ được so sánh cùng quỹ đạo; WSD đỉnh phẳng cho phép nối tiếp tự nhiên.

Code hoạt động thế nào (quota cap khi stream)

```
if phrase in fable.lower():
    if phrase_written >= cap_frac * (written + 1):  # vượt quota 10%?
        continue                                   # bỏ mẫu này
    has_phrase = True
...
written += 1
if has_phrase: phrase_written += 1
```

Bất biến của vòng lặp: tại mọi thời điểm $\text{phrase_written}/\text{written} \leq \sim \text{cap_frac}$; corpus kết quả có đúng tỉ lệ mong muốn mà không cần hai lượt quét.

Phản biện dự kiến

- “Cap 10% có làm mất thông tin?” Có chủ đích: đánh đổi một khuôn thừa mứa lấy đa dạng. Kiểm chứng hậu nghiệm bằng metric đa dạng (Distinct-2 sát truyện thật) và judge tăng.
- “Đây có phải data leakage/bias thủ công?” Là data CURATION có khai báo, tác động lên phân bố train, đo lường minh bạch trước/sau trên tập held-out không đổi.

8. M7: Hệ đo lường, và phát hiện quan trọng nhất về phương pháp

Tôi làm gì

Xây 3 tầng đo: (1) perplexity held-out so với “sàn lý thuyết”, (2) metric nội tại không cần tham chiếu (Distinct-1/2, Self-BLEU, Flesch, Zipf, loss theo vị trí), (3) LLM-as-judge 4 trục (grammar, creativity, moral_clarity, prompt_adherence; overall = trung bình). Sau đó TỰ ĐO NHIỀU của judge và thiết lập protocol n=45 seed bắt cặp.

Từng thước đo nghĩa là gì và vì sao chọn

- **Perplexity vs sàn e^1 (train loss)**: ppl held-out 3.56 so với $e^1.278 = 3.59$, chênh dưới 1%. Ý nghĩa: hành vi trên dữ liệu CHƯA THẤY khớp loss huấn luyện, tức không overfit. Đây là phép kiểm generalization rẻ nhất có thể.
- **Distinct-n**: tỉ lệ n-gram khác nhau trên tổng n-gram của TẬP truyện sinh; thấp nghĩa là lặp từ/cụm. **Self-BLEU**: BLEU của từng truyện so với các truyện còn lại trong cùng tập; cao nghĩa là các truyện na ná nhau (rập khuôn). **Flesch reading ease**: công thức tuyến tính trên độ dài câu và số âm tiết/từ; 80-100 là dài “dễ đọc”, khớp văn thiếu nhi. **Zipf**: phân bố tần suất-hạng từ vựng; truyện sinh bám đường Zipf của truyện thật nghĩa là dùng từ với “nhịp” tự nhiên. **Loss theo vị trí**: cross-entropy trung bình theo vị trí tương đối trong truyện; đường phẳng = không “đuối sức” giữa chừng.
- **Vì sao cần LLM-judge?** Các metric trên KHÔNG đo được “truyện có hay, có logic, có đúng đề không”. Judge (Qwen3-4B, rubric 4 trục, trả JSON điểm 0-10 kèm rationale) là proxy rẻ cho đánh giá người. Nhưng proxy thì phải ĐO SAI SỐ của nó, dẫn đến phát hiện dưới đây.

Phát hiện phương pháp luận: nhiều judge

Chấm lại CÙNG MỘT checkpoint hai lần (cùng protocol, cùng seed sinh): RAFT cho 7.38 và 7.82; baseline 7.73/7.82; GRPO 8.00/8.45. Kết luận: ở n=15 prompt, nhiều của judge cỡ ± 0.4 điểm, NGANG với các hiệu ứng tôi đang truy tìm. Hệ quả tôi áp dụng cứng: - Delta < 0.5 ở n=15: coi là nhiều, không kết luận. - So sánh quyết định: n=45, prompt held-out, **seed bắt cặp** (hai model sinh với cùng seed trên cùng prompt, so hiệu từng cặp rồi t-test trên hiệu). - Hai kết luận sớm bị RÚT LẠI theo quy tắc này: “DPO +5 điểm adherence” và “GRPO +0.45”.

Vì sao seed bắt cặp mạnh hơn: biến thiên do prompt (dễ khó/dễ) bị triệt tiêu khi lấy hiệu theo cặp, nên phương sai của ước lượng delta giảm mạnh, t-test nhạy hơn với cùng n.

Phản biện dự kiến

- “ $t=6.53$ nghĩa là gì?” Thống kê t bắt cặp: delta trung bình chia cho sai số chuẩn của delta ($sd/\sqrt{n}$). $t=6.53$ với $n=45$ tương ứng p cực nhỏ; so với ngưỡng ~ 2.0 của mức ý nghĩa 5%, kết quả 60M vượt xa mọi nghi ngờ nhiều.
- “Judge có bias gì?” Đã quan sát: chấm hào phóng hơn người đọc $\sim 1-1.5$ điểm ở sinh tự do, và có phương sai lớn giữa các lần chạy. Vì vậy điểm judge chỉ dùng SO SÁNH nội bộ cùng judge cùng protocol, không so chéo với điểm của judge khác (điểm E1 không so ngang điểm E3 chấm bằng judge khác được).

9. M8: Chiến dịch hậu huấn luyện 5 phương pháp (phần dễ bị hỏi xoáy nhất)

Bối cảnh: sau Phase 2, hạn chế còn lại là adherence $\sim 70\%$ và độ ổn định. Đã loại trừ sampling (temperature 0.7 và 0.8 cho adherence giống hệt nhau). Giả thuyết: alignment sẽ sửa được. Năm phương pháp được thử dưới CÙNG một protocol đánh giá.

9.1 DPO (Direct Preference Optimization): null 7.88 vs 8.02

Cách làm: sinh 2 truyện/prompt từ chính Phase 2, judge chấm cả hai, giữ cặp có chênh lệch ≥ 1.0 điểm thành (chosen, rejected); 194 cặp; train bằng DPOTrainer (TRL), beta 0.1, lr 5e-6, 2 epoch, có ref model đóng băng và guard perplexity.

DPO tối ưu cái gì: thay vì train một reward model rồi PPO, DPO tối ưu trực tiếp $-\log \text{sigmoid}(\text{beta} * [(\log \pi(yw|x) - \log \pi_{\text{ref}}(yw|x)) - (\log \pi(y_l|x) - \log \pi_{\text{ref}}(y_l|x))])$ tức là kéo GIÃN khoảng cách log-likelihood giữa bản được chọn (yw) và bản bị loại (yl), so với mô hình tham chiếu; beta điều tiết độ mạnh.

Code cốt lõi (dpo_train_local.py):

```
cfg = DPOConfig(num_train_epochs=2, learning_rate=5e-6, beta=0.1,
                 per_device_train_batch_size=4, gradient_accumulation_steps=2)
trainer = DPOTrainer(model=model, ref_model=ref, args=cfg,
                    train_dataset=pairs) # mỗi hàng: prompt, chosen, rejected
```

ref_model là bản sao Phase 2 ĐÓNG BẰNG: các log-ratio trong loss đều tính “so với ref” để mô hình không trôi tự do; trước và sau train đo perplexity held-out làm guard catastrophic forgetting (kết quả 0% drift).

Trong lúc train mọi thứ trông hoàn hảo: reward accuracy đạt 1.0 (mô hình xếp đúng chosen > rejected trên tập train), margin dương, ppl held-out 0% drift. **Nhưng judge eval trên prompt held-out: null.**

Vì sao null, cơ chế: chosen và rejected đều RÚT TỪ CÙNG MỘT MÔ HÌNH, chất lượng sàn sàn nhau (khác nhau chủ yếu do nhiễu sampling và nhiễu judge). Tín hiệu “tương đối” giữa hai bản gần giống nhau quá yếu để dịch phân bố; mô hình chỉ cần dịch chuyển tí xíu là phân biệt được cặp train (accuracy 1.0) mà không đổi hành vi tổng thể.

9.2 Best-of-N: +0.8, phương pháp DUY NHẤT ăn tiền, đã ship vào app

Câu hỏi phân định đặt ra trước: model KHÔNG THỂ viết hay, hay chỉ KHÔNG ỔN ĐỊNH? Thí nghiệm headroom: sinh K=3 ứng viên/prompt ở 3 nhiệt độ (0.5/0.8/1.1), judge chọn bản tốt nhất. Trung bình 1 mẫu 7.72; best-of-3 8.55; nhiều mẫu đơn lẻ 9.0-9.5 (Qwen-4B 9.75).

Kết luận bản chất: ràng buộc là PHƯƠNG SAI, không phải capacity. Phân bố của mô hình ĐÃ CHỨA truyện gần chất lượng tham chiếu ở đuôi phải; vấn đề là mass chưa dồn về đó. Phát hiện này định nghĩa lại mục tiêu mọi thí nghiệm sau: không phải dạy cái mới, mà dồn xác suất về mode tốt sẵn có.

Code trong app (app/main.py): nếu best_of_n > 1, backend lặp K lần gmeta(...) với seed+k, mỗi bản gọi judge.evaluate, chọn pick_best_index(scores), trả bản thắng kèm log điểm từng ứng viên vào Activity Log. Chi phí tuyến tính theo N (SLM sinh ~1 giây/ bản nên chấp nhận được).

9.3 RAFT (reward-ranked fine-tuning): null 7.60 vs 7.78

Cách làm: xây corpus 200 truyện mà TỪNG truyện đạt judge ≥ 9.0 (ngưỡng tuyệt đối, trung bình 9.22; tỉ lệ prompt đạt chỉ 23%), SFT với loss-mask điều kiện, lr 2e-5, 3 epoch.

Vì sao null, cơ chế (ba tầng): 1. Mẫu best-of-N vẫn là mẫu RÚT TỪ PHÂN BỐ CỦA CHÍNH MÔ HÌNH (in-distribution); SFT trên chúng chỉ tô đậm lại mode nó vốn thích. 2. Cán cân ngân sách: ~60k token truyện đặt cạnh prior pretraining 600M token, cú hích một phần vụn. 3. Quan trọng nhất: gradient của SFT KHÔNG CÓ THÀNH PHẦN ÂM. Không gì đẩy xác suất RA KHỎI các mode tầm thường; chỉ kéo lên vùng vốn đã cao.

9.4 Reward model 30M: rút cổng kiểm định (thí nghiệm phụ trợ)

Định train một scorer nhỏ thay judge (để RL rẻ): backbone 30M + linear head, 2 biến thể (hồi quy MSE trên 535 nhãn; pairwise Bradley-Terry $-\log \text{sigmoid}(r_{\text{chosen}} - r_{\text{rejected}})$ trên 164 cặp). Cổng đặt trước: Spearman ≥ 0.5 trên validation. Kết quả: 0.22 và pair-acc 46.7% (ngang tung đồng). Diễn giải: tín hiệu “chất lượng theo judge” (vốn nhiễu +/-0.4) không học được từ ~500 nhãn ở dung lượng biểu diễn 30M. Bài học: vòng tự cải thiện cần tín hiệu sạch, và tín hiệu sạch là thứ đắt.

9.5 GRPO-lite (REINFORCE + baseline theo nhóm): null ở ngân sách này (+0.09, n=45)

Vì sao thử: đây là lớp phương pháp DUY NHẤT có đủ hai thành phần còn thiếu: exploration (rollout MỚI mỗi bước, on-policy) và GRADIENT ÂM (advantage âm đẩy xác suất xuống). Cũng là nội dung Week 10 của môn học (policy gradient, REINFORCE, variance reduction bằng baseline).

Cách làm (grp_train.py): mỗi bước lấy 4 prompt x 4 rollout; judge chấm từng rollout làm reward; chuẩn hóa advantage TRONG NHÓM cùng prompt $(r - \text{mean})/\text{std}$; loss $-\text{advantage} * \text{mean_logprob}(\text{truyện}) + \text{beta_KL} * \text{KL}(\pi || \pi_{\text{ref}})$; 60 bước ~ 960 lần gọi judge ~ 5 giờ, lr $3e-6$ rồi $1e-5$, checkpoint-resume.

Code cốt lõi (grp_train.py), đúng thứ tự một bước:

```
stories, seqs = rollout(prompt, G=4)           # 1. sinh 4 bản MỚI từ policy hiện tại
rewards = [judge(s) for s in stories]         # 2. judge chấm từng bản (reward)
adv = (r - r.mean()) / (r.std() + 1e-4)       # 3. advantage chuẩn hóa trong nhóm
for (toks, cond_len), a in zip(seqs, adv):
    lp = log_prob trung bình của policy trên phần truyện
    kl = (log pi - log pi_ref) trung bình    # 4. đo độ lệch khỏi mô hình gốc
    loss = -a * lp + 0.05 * kl               # 5. REINFORCE + phạt KL
    (loss / (B * G)).backward()              # 6. cộng dồn gradient cả batch
clip_grad_norm_(1.0); opt.step()
```

Điểm cần nói được: gradient của $-a * lp$ chính là $\text{advantage} * \text{grad log pi}$ của định lý policy gradient; $a > 0$ kéo xác suất truyện đó lên, $a < 0$ ĐẨY XUỐNG (thành phần âm mà SFT/RAFT không có); mỗi bước đều lưu checkpoint + ppl guard mỗi 10 bước.

Từng thành phần vì sao có mặt: - Baseline theo nhóm: reward judge toàn 6-9 (toàn dương); nếu không trừ baseline thì mọi rollout đều được “khen”, gradient chỉ phóng to logit chung. Trừ trung bình nhóm: bản hơn trung bình được kéo lên, bản kém bị ĐẨY XUỐNG. Chia std để chuẩn hóa thang. - Phạt KL với mô hình gốc đóng băng: chống reward hacking và catastrophic drift; RL trên reward nhiều rất dễ chạy trốn khỏi vùng ngôn ngữ tốt.

Kết quả và cách đọc TRUNG THỰC: ở $n=15$ thấy +0.45 (mừng hụt); áp quy tắc nhiều, mở rộng $n=45$ seed bắt cặp: co về +0.09, $t=0.54$, thắng/hòa/thua 17/10/18. Chẩn đoán bằng KL: cuối run chỉ $\sim 1e-3$ nats/token, policy GẦN NHƯ CHƯA DỊCH. Kết luận đúng phạm vi: “GRPO ở ngân sách ~ 960 lần gọi judge chưa đủ để dịch phân bố”, KHÔNG phải “GRPO sai” (DeepSeek-R1 dùng cơ chế này với hàng nghìn bước và reward rẻ/sạch). Nút chặn thực tế: judge 15 giây một lần gọi.

9.6 Distillation từ teacher: ÂM -0.37, và vì sao kết quả âm này quý

Cách làm: Qwen3-4B sinh 600 truyện theo đúng prompt 5-slot (system prompt ép văn đơn giản 150-250 từ, lọc ≤ 400 token theo tokenizer SLM); SFT 2 epoch trên đó.

Khác biệt bản chất với 4 phương pháp trước: đây là tín hiệu OFF-DISTRIBUTION thật (văn của mô hình khác, không rút từ phân bố SLM). Bằng chứng nó “thật”: loss trên văn teacher 2.94 (so với 0.67 trên văn tự sinh: mô hình thấy văn teacher LA), ppl held-out drift +4.4% (mô hình DỊCH thật, lần đầu tiên trong campaign).

Nhưng dịch XUỐNG: 7.57 vs 7.94 (n=45, t=-1.55). Cơ chế: học trò 30M bắt chước văn phong bề mặt của teacher (nhịp câu, từ vựng) vượt quá capacity của nó, đánh mất độ trôi chảy bản địa đã tối ưu trên 600M token. Đây là failure mode “imitation học style, không học content” (Gudibande 2023). 180k token distill không dạy nổi cấu trúc mới, chỉ đủ bề cong văn phong.

Vì sao quý: ghép với 4 null tạo thành lập luận kín hai chiều: - dữ liệu trong-phân-bố: không dịch được (thiếu gradient âm, thiếu tín hiệu mới); - dữ liệu ngoài-phân-bố ở liệu SFT: dịch sai hướng (vượt capacity). Suy ra phân bố mặc định nằm ở TỐI ƯU CỤC BỘ do pretraining quyết định; đường tắt post-training rẻ không tồn tại; muốn nâng sản phải quay về pretraining. Đây chính là mệnh đề được 60M kiểm chứng thuận chiều ngay sau đó.

Bảng nhớ nhanh cho phân biện

Phương pháp	Tín hiệu	Exploration	Gradient âm	Kết quả	Một câu cơ chế
DPO	preference tương đối	không	ngầm	null	cặp cùng nguồn quá giống nhau
SFT-on-best	best trong batch	không	không	null	tô đậm mode sẵn có
RAFT	ngưỡng tuyệt đối 9.0	không	không	null	in-distribution + không có lực đẩy xuống
GRPO-lite	advantage nhóm	có	có	null (budget)	KL 1e-3, policy chưa kịp dịch
Distill	off-distribution	(của teacher)	không	ÂM	imitation trap, vượt capacity
Best-of-N	judge chọn	test-time	-	+0.8	khai thác đuôi phải của phân bố

10. M9: 60M, kiểm chứng thuận chiều kết luận

Tôi làm gì

Thiết kế và train from scratch 59.6M tham số: hidden 768 (tăng từ 512), 12 query head / 4 KV head, giữ 8 khối và FFN 2048, **seq 1024** (gấp đôi), tokenizer 12k giữ nguyên; dữ liệu FULL TF1 sau lọc + dedupe = 2.341.231 truyện = 934M token, KHÔNG lặp epoch; 10.000 bước (WSD dời điểm decay từ kế hoạch 15.000 xuống, quyết định thời gian); Colab T4, checkpoint-resume qua 4 phiên (2 lần runtime bị thu hồi + 1 lần hết quota, không mất quá 500 bước mỗi lần).

Quyết định và lý do

- **Scale chiều RỘNG (hidden 768) thay vì SÂU (thêm khối):** hidden lớn tăng dung lượng biểu diễn từng bước attention/FFN, nhắm thẳng vào điểm yếu conditioning; giữ 8 khối để tốc độ suy luận gần như cũ.
- **Seq 1024:** gỡ trần kiến trúc 512 đã ghi trong hạn chế của 30M (truyện dài, kiểm soát độ dài).
- **Full data không lặp epoch:** 934M token / 60M tham số ~ 15.7 token/tham số qua 10k bước thực học ~ hơi dưới Chinchilla nhưng mỗi token đều MỚI (không lặp), chất hơn lặp 4 epoch của 30M.
- **Corpus pack int16 + cache Drive:** 2.34M truyện không thể giữ dạng list Python; đóng gói phẳng int16 (~1.9GB) + offsets + condLens, build MỘT lần (~50 phút) rồi cache lên Drive; các phiên resume sau khôi phục trong ~2 phút.

Kết quả (thuộc số)

	30M-p2	60M
Loss cuối	1.278	1.058
PPL held-out	3.56	2.87
Judge n=45 bắt cặp	7.939	8.956 (+1.017, t=6.53; thắng/hòa/thua 36/5/4)
Adherence	7.87	9.11

Ba điểm phát biểu được: (1) đây là can thiệp ĐẦU TIÊN của toàn E1 nâng được phân bố mặc định, đúng chỗ chuỗi null chỉ ra; (2) 60M mặc định vượt cả 30M + best-of-3 (8.55); (3) adherence 9.11 phá trần ~70-80% mà

mọi alignment không lay chuyển nổi trên 30M, xác nhận trần đó là CAPACITY thật chứ không phải lỗi huấn luyện.

Lưu ý trung thực khi bị hỏi: 60M thay đồng thời 3 biến (capacity, context, dữ liệu) nên +1.0 điểm là tác động TỔNG HỢP, không tách được phần đóng góp từng biến. Đây là trade-off có chủ đích: mục tiêu bước này là kiểm chứng mệnh đề “đầu tư vào pretraining thì sàn tăng”, không phải ablation từng biến (đã ghi trong hướng phát triển).

11. M10: Tích hợp ứng dụng

- **Best-of-N trong app:** tham số `best_of_n` (1/3/5) trong request; backend sinh N bản, judge chấm, trả bản tốt nhất + log điểm từng ứng viên; UI SegmentedControl.
- **Tính trọn vẹn truyện:** ngân sách token theo mức độ dài đặt sát trần kiến trúc (400/440/460), `done_reason` phân biệt kết thúc thật (“stop”) với bị cắt (“length”), và cắt đuôi về câu hoàn chỉnh cuối khi bị cắt. Kết quả 30/30 truyện thử trọn vẹn.
- **Quick evaluation:** judge 4 trục + objective metrics (Distinct-1/2, Flesch) cho từng lần sinh, kèm tooltip giải thích tham số trong Observability.
- **Registry:** model phân phối cuối là `s1m-60m` (kèm `s1m-30m-p2`); GGUF q8 + Modelfile trong Drive `final-models/`.

12. E1 dưới lăng kính BÁO CÁO NHÓM: benchmark chung, các phép bóc tách, và cách trả lời về điểm 3.30

Phần này đối chiếu deep-dive với đúng những gì báo cáo nhóm (team report) trình bày về E1. Đây là vùng “nguy hiểm” nhất khi phản biện vì chứa các con số trông bắt lợi; nắm chắc cơ chế của từng con số thì chúng lại thành điểm cộng về độ trung thực khoa học.

12.1 Hai tầng đánh giá của nhóm (phải phân biệt được ngay)

- **Đánh giá NỘI BỘ:** mỗi hướng tự so sánh các BIẾN THỂ của mình (E1: v1 vs Phase 1 vs Phase 2 vs 60M...) bằng judge, đề và thang đo riêng. Mọi con số 2.5 / 6.0 / 7.0 / 8.96 của tôi thuộc tầng này. Báo cáo nhóm ghi rõ: điểm nội bộ KHÔNG dùng để xếp hạng E1-E5.
- **Đánh giá THỐNG NHẤT (vòng chung):** 5 hệ thống đại diện chạy CÙNG 25 đề, sinh 125 truyện, cố định seed và cấu hình giải mã, LÀM MÙ danh tính, chấm bằng CÙNG một giám khảo gemma-4-26b-a4b-it. Kết quả: E4 9.20, E5 8.44, còn E1-E3 trong khoảng 2.81-3.30; đại diện của tôi `s1m-60m` đạt **3.30/10**.

12.2 Vì sao 3.30 ở vòng chung trong khi 8.96 nội bộ: bốn nguyên nhân, xếp theo trọng số

1. **Lệch prompt contract (nguyên nhân chính, báo cáo nhóm nêu đích danh).** Mô hình được huấn luyện với “hợp đồng” đầu vào cố định: đề bài, xuống dòng, rồi token đặc biệt `<|story|>` báo hiệu “bắt đầu kể”. Runner của vòng chung KHÔNG truyền token này. Với mô hình from-scratch bé, prompt thiếu `<|story|>` là input NGOÀI PHÂN BỐ ngay từ ký tự đầu: nó chỉ từng thấy đúng một khuôn trong đời. Khác hẳn mô hình instruct 3B (E4/E5) đã gặp hàng triệu kiểu prompt nên “miễn nhiễm” với thay đổi giao diện. Bài học phát biểu được: **độ bền với prompt contract cũng là một phần chất lượng hệ thống, và mô hình càng nhỏ càng giòn ở điểm này.**
2. **Giám khảo và thang đo khác.** Gemma 26B với rubric khác, đề khác, cách chấm khác: điểm không cùng hệ quy chiếu với Qwen-4B nội bộ (chính tôi cũng đã chứng minh trong E1 rằng ngay MỘT giám khảo còn tự lệch +-0.4 giữa hai lần chấm).
3. **Đề khác.** 25 đề vòng chung không phải test.jsonl nội bộ.
4. **Một lần sinh, không best-of-N.** Vòng chung tính best-of-N là “lớp tìm kiếm lúc suy luận” và loại khỏi phép đo chất lượng một lần sinh của checkpoint: đại diện E1 mất đi +0.8 điểm mà app thực tế đang có.

Câu trả lời mẫu khi bị hỏi thẳng “8.96 hay 3.30, số nào đúng?”: cả hai đều đúng cho câu hỏi của chúng: 8.96 đo “can thiệp nào trong E1 tốt hơn can thiệp nào” trên đúng hợp đồng huấn luyện (kết luận: gói scale-up thắng); 3.30 đo “hệ đầu-cuối chuyển giao sang giao diện thống nhất tốt đến đâu” và bộc lộ một giới hạn triển khai thật: tôi chưa chứng minh được khả năng GIỮ chất lượng khi prompt contract thay đổi. Báo cáo nhóm viết đúng như vậy: “điểm 3.30 không phủ định các bóc tách nội bộ, nhưng bộc lộ một giới hạn triển khai”. Nếu được làm tiếp: augment prompt format lúc train (train với và không với <|story|>) hoặc để runner tôn trọng Modelfile template của artifact.

12.3 Ba mức tuân thủ điều kiện: khung khái niệm trung tâm của báo cáo nhóm

Báo cáo nhóm phân biệt ba mức, từ nông đến sâu: 1. **Nhắc lại trường đầu vào** (từ khóa nhân vật/bối cảnh có mặt trong truyện): mức “echo”. Adherence nội bộ của tôi (judge chấm + slot recall) chủ yếu đo quanh mức này và mức 2 nông: 60M đạt cao (9.11 nội bộ). 2. **Tổ chức diễn biến phù hợp TOÀN BỘ điều kiện:** cốt truyện thực sự bị chi phối bởi đề. Phép bóc tách “độ phủ” của vòng chung: tăng số trường từ 2 lên 5, đo phần độ phủ tăng thêm. **E1 chỉ +0.44/5** (thêm 3 trường mà truyện gần như không phủ thêm), trong khi E5 +3.68/5. Điểm nhất quán nội tại của E1 “gần như không đổi”: truyện TỰ hợp lý nhưng KHÔNG phụ thuộc đề bài. 3. **Counterfactual sensitivity:** đổi đúng MỘT điều kiện (tính cách/kết cục), kiểm tra truyện có đổi đúng hướng không. **E1: 1/10 cặp** (E5: 10/10). Đây là phép thử “mô hình có DỪNG điều kiện như nguyên nhân của diễn biến không” chứ không chỉ lặp từ khóa.

Cách hiểu đúng sự chênh giữa adherence nội bộ 9.11 và hai phép đo trên: chúng đo HAI CẤU TRÚC KHÁC NHAU. Judge nội bộ hỏi “truyện có bám các trường không” trên format đúng hợp đồng: 60M làm tốt. Vòng chung hỏi “điều kiện có làm thay đổi cốt truyện một cách nhân quả không” trên format lạ: 60M chưa làm được. Kết hợp cả hai: mô hình 60M đã học NHẮC và DẾT từ khóa vào truyện, nhưng năng lực SUY LUẬN NHÂN QUẢ THEO ĐIỀU KIỆN vẫn ngoài tầm với ở quy mô này (và bị đo trong điều kiện bất lợi vì lệch contract). Nói được câu này là qua được câu hỏi xoáy nhất của buổi bảo vệ.

12.4 E1 trả lời câu hỏi nghiên cứu nào của nhóm

- **RQ1 (năng lực nền):** E1+E2 là bằng chứng cho nhánh from-scratch: cải thiện tokenizer, thêm token, tăng quy mô đều nâng chất lượng NGÔN NGỮ, nhưng mô hình vẫn không dùng ổn định TỔ HỢP điều kiện: sai số của from-scratch nằm ở tầng suy luận điều kiện, còn sai số của E4/E5 (kế thừa nền 3B) chuyển sang tầng tuân thủ định dạng.
- **RQ2a (dữ liệu và tối ưu):** báo cáo nhóm ghi nhận cho E1: “mở rộng lên toàn bộ TF1 là can thiệp dương rõ nhất”.
- **RQ2b (đặt năng lực thích nghi ở đâu):** E1 tăng năng lực TOÀN mô hình (30M lên 60M) và thu lợi ích; E3 giữ nền đóng băng, mở adapter sang MLP và cũng thu lợi ích. Bài học ghép đôi: nền còn yếu thì tăng toàn mô hình; nền bị đóng băng thì mở rộng adapter.
- **RQ4 (kiểm soát lúc suy luận):** best-of-N của E1 và repair của E4 là HAI can thiệp suy luận có cải thiện đo được của cả nhóm; best-of-N giảm phương sai lựa chọn. Trong cùng phạm vi ngân sách, “nhiều thử nghiệm DPO, RAFT, GRPO, distillation, SFT và LoRA nhỏ không cải thiện hoặc làm giảm”: chuỗi null của tôi là một phần bằng chứng chung.

12.5 Các chi tiết kỹ thuật nhỏ trong báo cáo nhóm cần biết khi bị soi

- **Loss 1.447 / 1.278 là loss của CỬA SỐ LOG CUỐI** (trung bình các bước logging cuối), không phải trường train_loss tổng hợp cả run của HF Trainer: nếu bị hỏi vì sao số khác file log, trả lời được ngay.
- **Cách hiện thực metric trong mã E1** (báo cáo nhóm mô tả đúng): Distinct tính trên token tách theo KHOẢNG TRẮNG (không phải BPE token); Self-BLEU là trung bình precision 4-gram trên mọi cặp truyện; Flesch tính từng truyện rồi lấy trung bình. Vì phụ thuộc cách tách và cỡ mẫu, các giá trị này chỉ so trong cùng protocol.
- **Trạng thái artifact của E1 trong bảng kiểm kê nhóm:** có log xây corpus, có script + nhật ký huấn luyện, truyện sinh gồm “mẫu local + 25 attempt global”.
- **Giới hạn của so sánh 30M vs 60M** (báo cáo nhóm nhấn lại đúng điều tôi tự khai): tham số, context và ngân sách token đổi đồng thời, nên không suy ra “ngưỡng capacity phổ quát”; kết luận hợp lệ là CẢ GÓI scale-up hiệu quả hơn các cấu hình hậu huấn luyện đã thử.

13. Từ điển thuật ngữ cho người bắt đầu từ số 0

Đọc phần A trước: nắm được vòng lặp cốt lõi của GenAI thì mọi thuật ngữ sau đó chỉ là chi tiết của vòng lặp ấy.

A. GenAI trong 5 phút: vòng lặp cốt lõi

Mô hình ngôn ngữ (language model) là gì? Là một cỗ máy chỉ làm đúng MỘT việc: nhìn đoạn văn bản đã có, ĐOÁN chữ tiếp theo. “Con cáo nhảy qua...” nó đoán “hàng rào” với xác suất cao, “cái tủ lạnh” với xác suất thấp. Sinh cả một câu chuyện = lặp phép đoán này hàng trăm lần, mỗi lần lấy chữ vừa đoán nối vào đuôi rồi đoán tiếp.

“Huấn luyện” (training) nghĩa là gì? Cho máy đọc hàng triệu câu văn thật, mỗi lần che chữ tiếp theo và bắt nó đoán. Đoán lệch thì tính chính một chút hàng chục triệu “núm vặn” bên trong (gọi là THAM SỐ hay TRỌNG SỐ) theo hướng làm phép đoán bớt lệch. Lặp hàng triệu lần, các núm vặn dần “khắc” được ngữ pháp, từ vựng và cả lối kể chuyện của kho dữ liệu vào bên trong.

Bốn nhân vật trong mọi câu chuyện huấn luyện: - *Dữ liệu*: các câu văn mẫu để học (của tôi: 3 triệu truyện ngụ ngôn). - *Mô hình*: cỗ máy đoán, gồm kiến trúc (cách nối các phép tính) + tham số (các núm vặn). - *Loss*: con số đo “đoán lệch bao nhiêu”; huấn luyện = làm loss giảm. - *Optimizer*: thuật toán quyết định vặn núm nào, vặn bao nhiêu, sau mỗi lần đoán lệch.

From-scratch vs fine-tune: from-scratch = núm vặn khởi đầu HOÀN TOÀN NGẪU NHIÊN, máy chưa biết cả tiếng Anh (E1 của tôi). Fine-tune = lấy máy người khác đã dạy xong, chỉnh thêm cho việc của mình (các hướng E3-E5).

B. Chữ và số: cách máy “đọc” văn bản

- **Token**: máy không đọc từng chữ cái hay từng từ, mà đọc từng “mảnh chữ” gọi là token. Từ phổ biến (“the”, “fox”) là 1 token; từ hiếm bị bẻ thành vài mảnh (“unbelievable” -> “un/believ/able”). Truyện 250 từ của tôi cỡ 330-400 token.
- **Tokenizer**: cuốn “từ điển mảnh chữ” quy định bẻ văn bản thành token thế nào. Phải CHỐT trước khi huấn luyện và dùng y nguyên khi sinh.
- **BPE (Byte-Pair Encoding)**: cách tự học ra cuốn từ điển đó: bắt đầu từ từng ký tự, đếm cặp nào hay đứng cạnh nhau nhất thì dán thành một mảnh mới, lặp đến khi đủ số mảnh mong muốn (tôi chọn 12.000).
- **Vocab (vocabulary)**: tổng số mảnh trong từ điển. Vocab to thì mỗi câu ít token hơn nhưng tốn tham số cho bảng tra; vocab nhỏ thì ngược lại. Với mô hình bé, tôi chọn vocab nhỏ để dành tham số cho phần “suy nghĩ”.
- **Embedding**: bảng tra biến mỗi token thành một dãy số (vector) dài 512 (hoặc 768). Đây là “tọa độ ý nghĩa”: các từ hay xuất hiện trong ngữ cảnh giống nhau sẽ có tọa độ gần nhau sau huấn luyện. Tied embedding = dùng chung bảng này cho cả chiều đọc vào và chiều dịch từ suy nghĩ ra chữ, tiết kiệm một bảng to.
- **Token đặc biệt**: các “biển báo” tôi tự thêm: <|story|> (hết đề bài, bắt đầu truyện), <|end|> (hết truyện, dạy máy biết DỪNG), <|pad|> (miếng đệm cho đủ khuôn).
- **Context / sequence length**: số token tối đa máy nhìn được một lúc, như “tầm mắt”. 30M của tôi nhìn 512 token; quá tầm đó là không thấy. Đây là lý do truyện dài bị đuối và là thứ tôi nâng lên 1024 ở bản 60M.

C. Bên trong cỗ máy: kiến trúc transformer

- **Transformer**: kiểu kiến trúc thống trị GenAI hiện nay; đặc sản là cơ chế attention.
- **Attention (chú ý)**: khi đoán chữ tiếp theo, máy KHÔNG nhớ mù mờ cả đoạn, mà tính điểm “tôi nên nhìn lại chữ nào” cho TỪNG chữ phía trước, rồi trộn thông tin theo điểm đó. Ví dụ đoán đại từ cho nhân vật, attention sẽ “soi” ngược về chỗ tên nhân vật xuất hiện. Nhân quả (causal) nghĩa là chỉ được nhìn về trước, không nhìn tương lai.
- **Head (đầu chú ý)**: một “con mắt” attention. Nhiều head = nhiều con mắt nhìn theo nhiều tiêu chí song song (mắt này soi ngữ pháp, mắt kia soi nhân vật...).

- **GQA (Grouped-Query Attention):** mẹo tiết kiệm: 12 con mắt hỏi (query) nhưng dùng chung 4 cuốn sổ tra cứu (key-value) thay vì mỗi mắt một cuốn. Gần như không giảm chất lượng mà nhẹ bộ nhớ hẳn khi sinh văn bản.
- **RoPE:** cách báo cho máy biết THỨ TỰ các chữ (vì attention tự nó không biết trước sau). RoPE “xoay” tọa độ mỗi chữ một góc tỉ lệ vị trí; hai chữ càng xa nhau góc lệch càng lớn, nên máy cảm nhận được khoảng cách tương đối mà không tốn thêm nôm vãn nào.
- **RMSNorm:** trạm “ổn áp” giữa các tầng, co giãn dãy số về biên độ chuẩn để phép tính tầng sau không bị quá to hay quá nhỏ. Phiên bản gọn nhẹ của LayerNorm.
- **FFN / SwiGLU:** sau khi attention “thu thập thông tin”, FFN là trạm “tiêu hóa” thông tin đó (hai phép biến đổi lớn). SwiGLU là biến thể có thêm “van cổng” quyết định giữ bao nhiêu phần thông tin đi qua, biểu diễn khá hơn ở cùng kích cỡ.
- **Khối (layer/block):** một combo [ổn áp -> attention -> ổn áp -> FFN], có đường tắt cộng thẳng (residual) để tín hiệu không bị “tam sao thất bản” qua nhiều tầng. Mô hình của tôi xếp 8 combo như vậy.
- **Hidden size:** độ rộng dãy số chạy trong máy (512 ở 30M, 768 ở 60M). Rộng hơn = mỗi bước “nghĩ” được nhiều thứ hơn, đổi bằng nhiều tham số hơn.
- **Decoder-only:** kiến trúc chỉ có phần sinh (không có phần mã hóa riêng như máy dịch đời cũ); cả đề bài lẫn truyện nằm chung một dòng chữ, máy đọc đề rồi viết tiếp.

D. Huấn luyện: các khái niệm vận hành

- **Loss (cross-entropy):** thước đo “độ bất ngờ”. Máy gán xác suất cho chữ đúng càng cao thì loss càng thấp. Loss 1.058 của 60M nghĩa là trung bình máy chỉ còn “phân vân giữa ~2.9 phương án” mỗi bước (xem perplexity).
- **Perplexity (PPL):** e mũ loss; cách nói dễ hình dung của loss: “máy đang phân vân giữa mấy lựa chọn tương đương?”. PPL đo trên bài KIỂM TRA (dữ liệu chưa từng học) mới nói lên năng lực thật.
- **Gradient:** kim chỉ nam cho biết mỗi nôm vãn nên xoay chiều nào để loss giảm; tính bằng backpropagation (lan truyền ngược sai số từ đầu ra về từng nôm).
- **Optimizer / AdamW:** người cầm kim chỉ nam đi vãn nôm. AdamW khôn hơn cách vãn thô: nhớ trung bình hướng đi gần đây (dà) và độ rung của từng nôm để vãn nôm nào mạnh, nôm nào nhẹ. Weight decay là lực kéo nhẹ mọi nôm về 0 cho đỡ “học vẹt”.
- **Learning rate (LR):** cỡ bước mỗi lần vãn. To quá thì loạng choạng phá hỏng, nhỏ quá thì học cả năm không xong. $3e-3 = 0.003$.
- **Lịch LR / WSD:** kế hoạch thay đổi cỡ bước theo thời gian: khởi động rón rén (warmup), đi đều chân giai đoạn chính (stable), rồi bước nhỏ dần để “an cư” vào đáy (decay). Ưu điểm của WSD so với kiểu giảm liên tục: đoạn giữa phẳng nên muốn dừng sớm hay kéo dài chỉ cần dời chỗ bắt đầu đoạn decay.
- **Batch:** số mẫu học gộp chung một lần vãn nôm; gộp nhiều thì hướng vãn đỡ nhiễu. Gradient accumulation = GPU yếu không nhét nổi batch to, nên chia nhỏ, cộng dồn kim chỉ nam của từng phần rồi mới vãn một lần: kết quả y như batch to.
- **Epoch:** một lượt học qua TOÀN BỘ kho dữ liệu. Lặp 4 epoch = đọc cả kho 4 lần.
- **fp16:** lưu số bằng 16 bit thay 32 bit: nhanh gấp rưỡi-đôi, tốn nửa bộ nhớ, đổi lại phải cẩn thận số quá nhỏ bị tròn về 0 (đã có kỹ thuật loss scaling lo).
- **Gradient clipping:** cầu chì: nếu kim chỉ nam đột nhiên chỉ một cú vãn khổng lồ (batch dữ liệu dị), cắt bớt về mức trần 1.0 để không phá trọng số.
- **Checkpoint / resume:** ảnh chụp toàn bộ trạng thái (nôm vãn + trí nhớ của optimizer – đang ở bước nào) lưu định kỳ; máy chết thì nạp ảnh chụp chạy tiếp như chưa hề gián đoạn. Sống còn khi dùng GPU miễn phí hay bị thu hồi.
- **Overfitting (học vẹt):** thuộc lòng bài tập nhưng thi bài lạ thì trượt. Phát hiện bằng cách so loss trên bài tập với PPL trên bài kiểm tra: nếu PPL xấu hơn nhiều là vẹt. Mô hình của tôi PPL sát “sản lý thuyết” nên không vẹt.
- **Under-training:** ngược lại với vẹt: máy đủ não nhưng học quá ít bài. Chữa bằng thêm dữ liệu/thêm bước, không cần đổi não. Đây chính là “bệnh” của v1.
- **Scaling law:** quy luật thực nghiệm “cứ thêm tham số + thêm dữ liệu + thêm tính toán thì loss giảm theo đường cong lũy thừa dự đoán được”. Chinchilla: điểm chi tiêu hợp lý cỡ 20 token dữ liệu cho mỗi 1 tham số. Tôi dùng quy luật này như bác sĩ dùng biểu đồ tăng trưởng: nhìn đường loss biết “còn lớn được nữa hay đã kích trần”.

E. Dữ liệu có điều kiện: các khái niệm riêng của E1

- **Slot / điều kiện hóa (conditioning)**: đề bài 5 ô (nhân vật, bối cảnh, thử thách, kết cục, bài học). “Có điều kiện” nghĩa là máy sinh truyện THEO đề, không sinh tự do.
- **Loss masking (-100)**: khi chấm điểm đoán, BỎ QUA phần đề bài (chỉ chấm phần truyện). -100 là mã quy ước của PyTorch cho “ô này miễn chấm”. Nhờ vậy máy dồn toàn bộ việc học vào kể chuyện thay vì học thuộc mẫu đề.
- **Slot dropout**: lúc học, thỉnh thoảng che bớt ô đề bài để máy quen với việc người dùng bỏ trống ô; giống luyện đề với đủ kiểu đề khuyết.
- **Dedupe / held-out**: khử trùng lặp dữ liệu; và tách riêng một phần dữ liệu KHÔNG cho học để làm bài kiểm tra (held-out = “giữ ra ngoài”).
- **Template collapse**: máy nghiện một khuôn kể (“wise old owl” xuất hiện 28% trong dữ liệu nhưng 90% trong truyện máy sinh) vì phép lấy mẫu hút về mode mạnh nhất. Chữa tại nguồn: hạn ngạch 10% khi xây kho dữ liệu.

F. Đánh giá: làm sao biết truyện “tốt”

- **Distinct-1/2**: đếm tỉ lệ từ/cặp-từ KHÁC NHAU trong cả tập truyện sinh ra; thấp nghĩa là quanh quẩn vài từ.
- **Self-BLEU**: đo các truyện trong tập GIỐNG NHAU đến đâu; cao nghĩa là truyện nào cũng na ná nhau (rập khuôn).
- **Flesch reading ease**: công thức đo độ dễ đọc từ độ dài câu và độ dài từ; 80-100 là vùng “trẻ em đọc được”.
- **Zipf**: trong ngôn ngữ tự nhiên, từ phổ biến thứ k có tần suất tỉ lệ $\sim 1/k$ (đường thẳng trên trục log-log). Truyện máy sinh bám đường này = nhịp dòng từ giống người.
- **LLM-as-judge**: thuê một AI lớn hơn (Qwen-4B) làm giám khảo chấm truyện theo phiếu điểm 4 mục (ngữ pháp, sáng tạo, độ rõ bài học, bám đề), thang 10. Rẻ hơn thuê người, NHƯNG giám khảo máy cũng chấm lệch giữa các lần.
- **Nhiều (noise) của judge**: chấm cùng một bài hai lần ra hai điểm chênh tới 0.4-0.5. Tôi ĐO con số này thay vì tin mù quáng, rồi đặt luật: chênh lệch nhỏ hơn nhiều thì không được kết luận gì.
- **Seed**: hạt giống ngẫu nhiên; cùng seed thì phép lấy mẫu lặp lại y hệt, giúp so sánh công bằng và tái lập thí nghiệm.
- **Seed bắt cặp + paired t-test**: cho HAI mô hình làm CÙNG đề với CÙNG hạt giống, chấm rồi lấy hiệu điểm từng cặp bài; kiểm định thống kê trên dãy hiệu đó. Cách này triệt tiêu chuyện “dễ dễ khó khó”, nên phát hiện khác biệt nhạy hơn nhiều. t càng lớn (so mốc ~ 2) thì khác biệt càng chắc chắn không phải may rủi; 60M đạt $t=6.53$.

G. Hậu huấn luyện: các cách “dạy thêm” và vì sao khó

- **Sampling / temperature / top_p**: khi sinh chữ, máy không luôn chọn chữ xác suất cao nhất mà GIEO XÚC XẮC theo phân bố. Temperature chỉnh độ liều (cao = bay bổng, thấp = an toàn); top_p chỉ gieo trong nhóm chữ khả dĩ nhất. Đây là núm chỉnh LÚC SINH, không đổi trọng số.
- **SFT (supervised fine-tuning)**: dạy thêm bằng ví dụ mẫu: đưa các bài văn “chuẩn” và bảo máy học tiếp như hồi pretrain, chỉ khác là ít bài và có chọn lọc.
- **Preference / DPO**: dạy bằng SO SÁNH thay vì đáp án: đưa từng cặp (bài tốt hơn, bài kém hơn) và tinh chỉnh để máy tự tin hơn vào bài tốt. DPO là công thức toán làm điều đó trực tiếp trên trọng số, kèm một “mỏ neo” (mô hình gốc đóng băng) để không trôi quá xa.
- **RLAIF**: dùng AI giám khảo thay con người để tạo các nhãn tốt/kém ở trên.
- **Reward / reward model**: điểm thưởng cho một bài sinh ra; reward model là một máy chấm nhanh học từ các nhãn của giám khảo (của tôi rút kiểm định vì ít nhãn + nhãn nhiều).
- **RL / policy / REINFORCE**: học tăng cường: máy (policy) TỰ sinh bài, nhận điểm thưởng, rồi tự chỉnh để hành vi được thưởng xảy ra thường hơn. REINFORCE là công thức gốc: “tăng xác suất những gì vừa làm, tỉ lệ với điểm thưởng nhận được”.
- **Baseline / advantage**: nếu điểm thưởng lúc nào cũng dương, mọi hành vi đều được khen, chẳng học được gì. Trừ đi mốc trung bình (baseline) để bài hơn trung bình được kéo lên, bài KÉM trung bình bị đẩy xuống; phần chênh đó gọi là advantage.

- **GRPO**: biến thể REINFORCE dùng ngay trung bình của NHÓM bài cùng để làm mốc, khỏi cần huấn luyện thêm một máy ước lượng mốc.
- **KL penalty**: đây xích buộc máy đang học RL vào mô hình gốc: đi xa quá (phân bố lệch nhiều) là bị phạt, tránh việc máy “lách luật ăn điểm” mà hồng tiếng Anh.
- **On-policy / off-policy**: học từ bài do CHÍNH MÌNH vừa sinh (on) hay từ nguồn khác (off). In-distribution / off-distribution: dữ liệu thuộc “vùng quen” của máy hay ngoài vùng đó. Chuỗi null của tôi tóm gọn: dữ liệu vùng quen không tạo thay đổi, dữ liệu ngoài vùng thì máy bé không tiêu hóa nổi.
- **Best-of-N**: không dạy gì thêm: cho máy viết N bản, giám khảo chấm, lấy bản hay nhất. Ăn vào tính KHÔNG ỔN ĐỊNH của máy bé (viết 3 bản thể nào cũng có bản khá).
- **Knowledge distillation**: trò học từ thầy: cho máy bé học lại văn do máy lớn viết. Imitation trap: trò bé bắt chước GIỌNG thầy mà không đủ não hiểu Ý, kết quả mất luôn giọng tự nhiên của mình (đúng thứ tôi đo được: điểm giảm 0.37).
- **Catastrophic forgetting**: dạy thêm việc mới làm quên việc cũ; canh chừng bằng cách đo PPL trên bài kiểm tra cũ trước/sau khi dạy thêm.

H. Triển khai: từ trọng số đến ứng dụng

- **Quantization / GGUF / q8**: nén trọng số từ 16-32 bit xuống 8 bit (q8) cho file nhỏ chạy nhanh trên laptop, mất chất không đáng kể; GGUF là định dạng file cho công cụ llama.cpp/Ollama.
- **Ollama / Modelfile**: phần mềm chạy model cục bộ; Modelfile là “công thức nạp” khai báo file trọng số + tham số sinh mặc định + token dừng.
- **token/giây (tok/s)**: tốc độ sinh. slm-60m ~900 tok/s trên laptop, so ~19 tok/s của mô hình 4B cùng máy: lợi thế cốt lõi của mô hình nhỏ.
- **done_reason**: cờ báo lý do dừng khi sinh: “stop” = máy tự kết thúc truyện đang hoang (gặp <|end|>), “length” = bị đứt vì chạm trần số token (truyện cụt, cần xử lý).

14. Ngân hàng câu hỏi phản biện tổng hợp (trả lời trong 2-4 câu)

1. **“Vì sao chọn from-scratch thay vì fine-tune như các bạn khác?”** Vì câu hỏi nghiên cứu của tôi là về SẢN chất lượng và cái gì quyết định nó; chỉ from-scratch mới kiểm soát được mọi biến (tokenizer, dữ liệu, ngân sách) để quy kết quả về đúng nguyên nhân. Các hướng fine-tune trả lời câu hỏi khác (kể thừa prior thì thêm được gì).
2. **“Kết quả 4 phương pháp null có phải do làm sai không?”** Ba bằng chứng nói không: (a) tín hiệu trong-train của từng phương pháp đều đạt (DPO accuracy 1.0, RAFT loss giảm, GRPO chạy đúng cơ chế advantage); (b) mọi so sánh dùng protocol cố định seed bắt cặp, có đo nhiễu; (c) khi đầu tư đúng chỗ (60M) thì CÙNG protocol đó cho +1.0 điểm, chứng tỏ thước đo đủ nhạy để thấy hiệu ứng thật.
3. **“Null thì đóng góp gì?”** Nó khoanh vùng cơ chế: phân bố mặc định là tối ưu cục bộ của pretraining; kèm một phát hiện phương pháp luận độc lập (nhiều judge +-0.4 và quy trình xử lý nó). Chuỗi null chỉ đường cho quyết định 60M, và 60M xác nhận.
4. **“Vì sao best-of-N không phải gian lận?”** Nó là inference-time compute có khai báo, cùng họ với self-consistency/rejection sampling trong văn liệu; báo cáo tách bạch điểm mặc định (7.94) và điểm best-of-3 (8.55), không trộn.
5. **“PPL 2.87 của bạn so với PPL của nhóm khác được không?”** Không: khác tokenizer (12k vs 49k), khác tập held-out, khác cách mask. PPL chỉ so trong cùng hệ quy chiếu; so chéo hệ thống phải dùng cùng judge cùng đề (như bảng benchmark chung của nhóm).
6. **“60M ăn +1.0 do biến nào?”** Không tách được, ba biến đổi cùng lúc (capacity, context, data); đó là thiết kế kiểm chứng mệnh đề tổng, ablation từng biến là việc tương lai. Có thể nói thêm: adherence tăng mạnh nhất gợi ý capacity + context đóng vai trò lớn, vì độ đúng là hai thứ 30M bị chặn.
7. **“Nếu có thêm 1 tuần GPU, làm gì tiếp?”** Theo thứ tự bằng chứng: (a) ablation 60M (giữ data, chỉ đổi capacity) để tách biến; (b) distillation ở quy mô PRETRAIN (trộn token teacher vào corpus) thay vì SFT; (c) GRPO với reward rẻ (rule-based: slot recall, completeness) để mua đủ số bước.
8. **“Vòng chung của nhóm chấm slm-60m có 3.30/10, giải thích thế nào?”** Ba lớp: (a) nguyên nhân chính là lệch prompt contract: mô hình train với token <|story|>, runner chung không truyền nó, nên input ngoài phân bố ngay từ đầu; (b) giám khảo, thang đo, đề đều khác nên số không

cùng hệ quy chiếu với 8.96 nội bộ; (c) tôi nhận đây là giới hạn triển khai THẬT: mô hình bé from-scratch giòn với thay đổi giao diện, và nêu hướng sửa (augment format lúc train / runner tôn trọng template artifact).

9. **“Counterfactual 1/10 nghĩa là mô hình của em không hiểu điều kiện?”** Nó nghĩa là mô hình chưa dùng điều kiện làm NGUYÊN NHÂN của diễn biến (mức 3 trong khung ba mức tuân thủ); mức 1-2 (nhắc và dẹt từ khóa trên format đúng hợp đồng) thì adherence nội bộ 9.11 cho thấy đạt. Phân biệt được hai cấu trúc đo này chính là điều báo cáo nhóm muốn làm rõ; và phép đo counterfactual của E1 còn chịu thêm bất lợi prompt contract.
10. **“Điểm nội bộ của các nhóm có so được với nhau không?”** Không, và báo cáo nhóm cấm điều đó một cách tường minh: khác đề, khác giám khảo, khác thang. So ngang hàng chỉ dùng 125 truyện vòng chung. Chính E1 cung cấp cơ sở phương pháp cho quy tắc này bằng phép đo nhiều judge +0.4.
11. **“Điểm yếu lớn nhất của E1?”** Đánh giá phụ thuộc một judge có nhiều (đã đo và bù bằng $n=45$ bắt cặp nhưng không triệt tiêu); và 60M chưa được ablation tách biến. Tôi chủ động ghi cả hai trong báo cáo.

15. Ba mạch kể để mở đầu câu trả lời bất kỳ

1. **Mạch nhân quả:** chẩn đoán under-training -> cấp token -> sửa phân bố dữ liệu -> loại trừ sampling -> thử 5 alignment (null/âm) -> kết luận cơ chế -> scale 60M xác nhận.
2. **Mạch phương pháp:** mọi khẳng định đi kèm một phép đo có kiểm soát; khi thước đo nhiều thì đo chính thước đo; hai kết luận sai được rút lại công khai.
3. **Mạch trade-off mô hình nhỏ:** vocab nhỏ vì embedding, GQA vì cache, tied vì tham số, seq 512 vì $O(n^2)$, best-of-N vì phương sai: mỗi lựa chọn đều là một cân đối tài nguyên có thể giải thích bằng một câu.

16. Tóm tắt nhanh M1 -> M9 (kèm định nghĩa term, để ôn cấp tốc)

Mạch xuyên suốt: **chẩn đoán bệnh -> chữa bằng dữ liệu -> đo cho chuẩn -> thử mọi cách nâng điểm rẻ (đều thất bại) -> kết luận phải đầu tư pretraining -> scale up xác nhận.**

M1 - Tokenizer BPE 12k

Xây “bảng chữ cái” riêng cho model. - **Token** (mảnh chữ; model đọc token chứ không đọc từ/chữ cái) - “the”, “fox” = 1 token. - **BPE** (Byte-Pair Encoding: thuật toán tự học ra bộ token bằng cách ghép dần cặp ký tự hay đứng cạnh nhau). - **Vocab** (tổng số token trong bảng) = 12.000. - **Vì sao vocab nhỏ:** bảng **embedding** (bảng tra biến token thành vector số) tốn vocab $\times$ hidden; 12k tốn 6.1M (~17% của 30M), 50k của GPT-2 tốn 25.6M nuốt gần hết ngân sách.

M2 - Pipeline dữ liệu điều kiện 5-slot

- **5 slot** (5 ô đề: nhân vật, bối cảnh, thử thách, kết cục, bài học). Chuỗi: `<5 slot> <|story|> <truyện> <|end|>`.
- **Loss-mask -100** (gán nhãn -100 cho phần đề để bỏ qua khi chấm; model chỉ bị chấm ở phần truyện) -> dồn sức học vào kể chuyện.
- **Slot dropout** (che ngẫu nhiên từng ô đề lúc train, để model quen khi người dùng bỏ trống ô).
- **Dedupe** (khử truyện trùng lặp) + lọc 60-320 từ.

M3 - Kiến trúc Llama 30M

- **Decoder-only** (chỉ đọc chữ đã có -> đoán chữ tiếp theo), kiểu Llama.
- 8 **khối** (tầng xử lý xếp chồng), **hidden** (độ rộng vector mỗi token) 512, **FFN** (trạm “tiêu hóa thông tin”) 2048 dùng **SwiGLU** (FFN có “van cổng”).
- **Attention** (cho mỗi từ “nhìn lại và lấy thông tin từ các từ liên quan”): 8 **query head** (con mắt hỏi) / 2 **KV head** = **GQA** (nhiều mắt hỏi dùng chung ít số tra, tiết kiệm bộ nhớ).

- **RoPE** (bảo thứ tự chữ bằng phép xoay), **RMSNorm** (trạm ổn áp), **tied embedding** (dùng chung bảng embedding vào/ra).
- **from-scratch** (khởi tạo trọng số NGẪU NHIÊN). Tổng **36.6M tham số**.

M4 - Vòng lặp huấn luyện

- **AdamW** (thuật toán vận nùm khôn: nhớ đà + độ rung từng nùm), **weight decay** (kéo nùm về 0 chống học vệt), **gradient clip** (cầu chì chặn cú vẩy quá lớn).
- **Learning rate** (cỡ mỗi bước vận) đỉnh $3e-3$.
- **WSD** (lịch LR: warmup rón rén -> stable đi đều -> decay bước nhỏ dần; đoạn giữa phẳng nên dừng sớm/kéo dài chỉ cần dời điểm decay).
- **fp16** (số 16-bit cho nhanh/nhẹ), batch hiệu dụng 128 nhờ **gradient accumulation** (cộng dồn nhiều nhóm nhỏ rồi vận một lần).
- **Checkpoint + auto-resume** (lưu ảnh chụp trạng thái; máy chết nạp lại chạy tiếp).

M5 - Chẩn đoán under-training + Phase 1

- **v1**: cố ý ít dữ liệu (150k, 900 **bước/step**) -> judge 2.5. **Phase 1**: CHỈ tăng dữ liệu (400k, 1800 bước, ~600M **token** qua 4 **epoch/lượt**) -> judge 6.0.
- **Under-training** (não đủ nhưng học quá ít; chữa bằng thêm dữ liệu).
- **Chinchilla** (~20 token/tham số): 30M cần ~700M, v1 chỉ 1.7 -> thiếu 10 lần.
- **Muennighoff** (lặp tới ~4 lần gần bằng dữ liệu mới). **Kaplan power-law** (loss giảm lũy thừa; log-log thẳng = còn train được) $R^2=0.96$, chưa **plateau** (bệt ngang).
- Bài học: quy mô nhỏ, dữ liệu quyết định, kiến trúc thứ yếu.

M6 - Phase 2: can thiệp phân bố dữ liệu

- **Template/mode collapse** (model dồn xác suất vào MỘT khuôn; “wise old owl” 28% data -> 90% sinh). **Sampling** (cách gieo xúc xắc chọn chữ) khuếch đại mode mạnh.
- Hạn ngạch owl xuống 10% -> sinh còn 23%; hạ slot dropout teaching/outcome 0.30->0.15 -> bám bài học yêu cầu.
- **Resume** từ bước 1800 -> 3600. Kết quả owl 23%, judge 7.0, loss 1.278, ppl 3.56.
- Bài học: sửa lỗi khuôn TẠI DỮ LIỆU sạch hơn tại sampling (sampling phạt cả tên nhân vật -> **entity drift**).

M7 - Hệ đo lường + phát hiện nhiễu judge

- **Perplexity** (e mũ loss) trên **held-out** (dữ liệu tách riêng làm bài kiểm tra) -> 3.56 sát sà = không **overfit**.
- Metric **reference-free** (không cần đáp án mẫu): **Distinct** (đa dạng từ), **Self-BLEU** (truyện có na ná nhau không), **Flesch** (dễ đọc), **Zipf** (nhịp dừng từ).
- **LLM-as-judge** (model lớn Qwen-4B làm giám khảo chấm rubric 4 trục).
- **PHÁT HIỆN**: chấm CÙNG model 2 lần ra 2 điểm khác (RAFT 7.38/7.82...) -> **nhiều judge +-0.4 ở n=15** (n = số prompt chấm).
- Quy tắc: chênh <0.5 = nhiễu; quyết định dùng **n=45 seed bắt cặp** (2 model cùng đề cùng seed, lấy hiệu từng cặp, **paired t-test**); **t** >2 mới tin.

M8 - Chiến dịch hậu huấn luyện (5 phương pháp)

Thử 5 cách **post-training** (dạy thêm sau pretraining): - **DPO** (dạy bằng SO SÁNH cặp tốt/kém, neo model gốc) -> null. - **Best-of-N** (sinh N bản, giám khảo chọn bản hay nhất; không train) -> **+0.8, ship vào app**. - **RAFT** (lọc truyện tự sinh ≥ 9.0 rồi dạy lại) -> null. - Reward model (scorer nhỏ thay giám khảo) -> rớt cống. - **GRPO** (học tăng cường: model tự sinh, nhận điểm, tự chỉnh; có lực đẩy XUỐNG bản kém) -> null (60 bước quá ít). - **Distillation** (học lại văn teacher Qwen-4B viết) -> **ÂM** (bất chức hồng giọng). - Kết luận: phân bố mặc định ở **local optimum** (điểm tốt nhất trong vùng lân cận, không phải tốt nhất tuyệt đối) do pretraining quyết định -> không có đường tắt hậu huấn luyện rẻ.

M9 - Scale up 60M (kiểm chứng)

- from-scratch 59.6M: hidden 512->768, 12 query/4 KV head, giữ 8 khối, **seq 512->1024** (gỡ trần độ dài), tokenizer 12k giữ nguyên.
- FULL TF1: 2.34M truyện = 934M token, **KHÔNG lặp epoch** (mỗi token đọc đúng 1 lần).
- 10.000 bước (WSD dời điểm decay từ 15.000 vì hạn thời gian), Colab T4, checkpoint-resume qua 4 phiên (runtime **bị thu hồi**/hết quota, mất <=500 bước mỗi lần).
- Kết quả: loss 1.058, ppl 2.87, **judge 7.94->8.96 (+1.017, t=6.53, thắng 36/45)**, adherence 7.87->9.11.
- Ý nghĩa: can thiệp ĐẦU TIÊN nâng điểm mặc định - đúng dự đoán; adherence phá trần ~70-80% -> trần là **capacity** (dung lượng não) thật.
- Lưu ý: 60M đổi đồng thời 3 biến (capacity + context + data) nên +1.0 là tác động TỔNG HỢP, chưa tách riêng từng biến.

Một câu kết cho toàn bộ

Sản chất lượng của model nhỏ do pretraining (dữ liệu + token + capacity) quyết định; 5 phương pháp hậu huấn luyện rõ không nâng được sản, best-of-N khai thác được phần đuôi tốt lúc sinh, và scale lên 60M xác nhận đầu tư đúng chỗ là pretraining.

17. Hai cơ chế judge: judge nội bộ vs judge tổng (vòng chung)

Cả hai đều là LLM-as-judge (dùng một mô hình ngôn ngữ lớn làm giám khảo chấm điểm truyện, thay cho thuê người chấm), nhưng khác nhau ở gần như mọi thông số, nên điểm của hai bên KHÔNG so trực tiếp với nhau được.

Bảng đối chiếu

Yếu tố	Judge NỘI BỘ (của tôi)	Judge TỔNG (vòng chung của nhóm)
Giám khảo	Qwen3-4B	Gemma-4-26b-a4b-it (model lớn hơn)
Mục đích	so các biến thể E1 với nhau (v1/Phase 1/Phase 2/60M)	xếp hạng 5 hướng E1-E5 cạnh nhau
Tập đề Blind?	test.jsonl riêng của E1 không (chỉ so nội bộ)	25 đề chung, cố định có (làm mù danh tính model)
Prompt contract	đúng như lúc train (có < story >)	runner chung KHÔNG truyền < story >
Điểm slm-60m	8.96	3.30

Định nghĩa vài term trong bảng: - **Blind** / làm mù (giám khảo không biết đang chấm truyện của hướng nào, để tránh thiên vị vô thức). - **Prompt contract** (hợp đồng định dạng đầu vào: model được train để LUÔN nhận đúng một khuôn - đề bài, xuống dòng, rồi token <|story|> báo “bắt đầu kể”). - Tập đề held-out (bộ prompt tách riêng để chấm, model chưa từng thấy lúc train).

Mỗi judge trả lời câu hỏi gì

- **Judge nội bộ** trả lời: “trong các can thiệp của RIÊNG E1, cái nào tốt hơn cái nào?”. Vì chỉ so các model của tôi với nhau nên chỉ cần giữ cố định mọi thứ khác (cùng giám khảo, cùng đề, cùng cách chấm) là công bằng. Mọi con số 2.5 / 6.0 / 7.0 / 8.96 thuộc tầng này. Báo cáo nhóm ghi rõ: điểm nội bộ KHÔNG dùng để xếp hạng E1-E5.
- **Judge tổng** trả lời: “đặt cạnh nhau trên CÙNG một sân, 5 hướng của cả nhóm xếp hạng thế nào?”. Cần một sân chung duy nhất: cùng 25 đề, cùng giám khảo Gemma, blind, cùng giao diện triển khai.

Vì sao KHÔNG so hai điểm với nhau được

Hai judge khác **hệ quy chiếu (khung/thang tham chiếu, như đo bằng thước mét so với thước inch)**: 1. Giám khảo khác (Qwen-4B vs Gemma-26B): mỗi model chấm gắt/nới khác nhau. 2. Đề khác, cách chấm khác. 3. Chính tôi đã chứng minh ở M7: ngay MỘT giám khảo còn tự lệch ± 0.4 giữa hai lần chấm cùng bài (nhiều judge). Hai giám khảo khác nhau thì lệch còn nhiều hơn.

Vì vậy 8.96 và 3.30 KHÔNG mâu thuẫn: chúng là hai phép đo trả lời hai câu hỏi khác nhau bằng hai cây thước khác nhau.

Vì sao slm-60m tụt mạnh ở vòng chung (3.30)

Nguyên nhân CHÍNH không phải model dở, mà là lệch **prompt contract**: - Runner vòng chung không truyền `<|story|>`. Với model **from-scratch (huấn luyện từ đầu, chỉ từng thấy đúng một khuôn prompt trong đời)** bé, prompt thiếu `<|story|>` là input **out-of-distribution (ngoài phân bố: dạng dữ liệu model chưa từng gặp lúc train)** ngay từ ký tự đầu. - Model instruct 3B (E4/E5) đã thấy triệu kiểu prompt nên “miễn nhiễm” với thay đổi giao diện; model của tôi thì “giòn”.

Bài học phát biểu được: độ bền với thay đổi prompt cũng là một phần chất lượng hệ thống, và model càng nhỏ càng giòn ở điểm này. Đây là giới hạn triển khai thật của E1; hướng sửa: train model với CẢ hai dạng (có và không `<|story|>`) để nó quen, hoặc để runner tôn trọng template của artifact.

Một câu tóm tắt: judge nội bộ đo “can thiệp nào trong E1 tốt hơn” trên đúng sân nhà; judge tổng đo “5 hướng xếp hạng thế nào” trên sân chung - khác thước nên không so điểm trực tiếp, và điểm 3.30 chủ yếu phản ánh model chưa quen giao diện lạ chứ không phải năng lực kém.